FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA

Arthur dos Santos Cabral, 566515, 2TDSA

Bruno Martins Bettio, 564939, 2TDSA

José Diogo da Silva Neves, 562341, 2TDSA

Júlia Tiziotto Buttler, 564975, 2TDSA

Mariana Xavier Quispe, 566357, 2TDSA

[Code & Paws]

São Paulo

2026

FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA

Arthur dos Santos Cabral, 566515, 2TDSA

Bruno Martins Bettio, 564939, 2TDSA

José Diogo da Silva Neves, 562341, 2TDSA

Júlia Tiziotto Buttler, 564975, 2TDSA

Mariana Xavier Quispe, 566357, 2TDSA

[Code & Paws]

Challenger, sprint 1 e 2 apresentadas à Faculdade de Informática e Administração Paulista como requisito de nota para a avaliação da disciplina Mastering Relational and Non-Relational Datababase, sob a orientação do Professor Marcel Thomé Filho.

São Paulo

2026

SUMÁRIO

1. DESCRIÇÃO DO PROJETO
   1.1 Objetivo da Solução
   1.2 Justificativa da Escolha do Tema
   1.3 Tecnologias Utilizadas
   1.4 Informações a serem Persistidas
2. DOCUMENTAÇÃO POR DISCIPLINA
   2.1 Building Relational Database
   2.1.1 Entidade USERS
   2.1.2 Entidade VETERINARIOS
   2.1.3 Entidade PETS
   2.1.4 Entidade PET_VETERINARIO
   2.1.5 Entidade DIARIO_ENTRADAS
   2.1.6 Entidade REGISTROS
   2.1.7 Entidade COMENTARIOS
   2.1.8 Entidade INSIGHTS
   2.1.9 Entidade SOLICITACOES
   2.1.10 Entidade LOGS_ERROS
   2.1.11 Relacionamentos e Cardinalidade

2.2 Procedures e Funcionalidades Implementadas
   2.2.1 Procedure de Log de Erros
   2.2.2 Procedures de Inserção (Carga de Dados)
   2.2.3 Tratamento de Exceções
   2.2.4 Funcionalidades Implementadas
   2.2.5 Relatórios e Consultas (Blocos Anônimos)
      2.2.5.1 Relatório 1 – Pets por Tutor
      2.2.5.2 Relatório 2 – Insights por Veterinário
      2.2.5.3 Relatório 3 – Análise de Registros (LAG/LEAD)
      2.2.5.4 Relatório 4 – Classificação de Registros
3. SPRINT 3 – NOVAS FUNCIONALIDADES
4. SPRINT 4 – Empacotamento
5. MODELAGENS (Oracle SQL Developer Data Modeler)
      5.1 Modelo Lógico
      5.2 Modelo Relacional
      5.3 Scripts DDL

1 DESCRIÇÃO DO PROJETO

1.1 Objetivo da Solução:

O projeto tem como objetivo desenvolver uma plataforma digital voltada para o acompanhamento contínuo da saúde animal, transformando o cuidado com pets em um processo estruturado, preventivo e integrado ao acompanhamento veterinário.

A solução propõe a criação de um “diário inteligente do pet”, onde tutores registram informações do dia a dia do animal, como alimentação, comportamento, atividades e sintomas. Esses registros são organizados cronologicamente e centralizados em um histórico único por pet.

Além disso, o sistema permite a integração entre tutores e veterinários, possibilitando que o profissional acompanhe a evolução clínica do animal, realize observações e forneça orientações diretamente na plataforma. O sistema também prevê o uso de inteligência artificial para análise dos dados e geração de insights automáticos sobre possíveis alterações no estado de saúde do pet.

1.2 Justificativa da Escolha do Tema:

O tema foi escolhido devido a um problema recorrente no cuidado com animais de estimação: a falta de organização e histórico estruturado sobre a saúde dos pets.

Muitos tutores realizam apenas cuidados reativos, ou seja, procuram ajuda veterinária apenas quando o problema já está avançado. Isso dificulta o diagnóstico precoce e impede a identificação de padrões de comportamento e saúde ao longo do tempo.

Além disso, a comunicação entre tutor e veterinário muitas vezes depende exclusivamente da memória do responsável pelo animal, o que pode gerar informações incompletas ou imprecisas.

Com isso, a proposta do sistema busca centralizar essas informações em um único ambiente, permitindo acompanhamento contínuo, histórico detalhado e suporte inteligente baseado em dados. Isso melhora tanto a qualidade do cuidado preventivo quanto a tomada de decisão clínica.

1.3 Tecnologias Utilizadas:

- Oracle SQL (Banco de dados relacional)
- PL/SQL (Procedures, cursors e lógica de dados)
- DBMS_OUTPUT (Relatórios e visualização de dados)
- SQL (DDL e DML)
- Conceitos de modelagem relacional (MER e normalização)

1.4 Informações a serem persistidas:

PK (Primary Keys):

- id_users
- id_veterinario
- id_pet
- id_pet_veterinario
- id_diario_entrada
- id_registro
- id_comentario
- id_insight
- id_solicitacao
- id_log_erro

FK (Foreign Keys):

- user_id (em pets, veterinarios, comentarios, solicitacoes)
- pet_id (em diario_entradas, pet_veterinario, insights, registros, solicitacoes)
- veterinario_id (em pet_veterinario, solicitacoes)
- entrada_id (em registros, comentarios)

Essas chaves garantem integridade referencial e permitem consultas como:

- “Quais pets pertencem a um tutor específico?”
- “Quais registros diários indicam alterações no comportamento?”
- “Quais insights foram gerados para determinado pet?”
- “Qual veterinário está responsável por cada pet?”
- “Quais solicitações estão pendentes ou aceitas?”

2 DOCUMENTAÇÃO POR DISCIPLINA

2.1 BUILDING RELATIONAL DATABASE

Para garantir a estrutura lógica e escalável do sistema, foram definidas as seguintes entidades principais:

1. USERS

Tabela responsável pelo cadastro de usuários do sistema (tutores e veterinários).

Campos:

- id (PK)
- nome
- email
- senha
- telefone
- tipo_usuario (TUTOR ou VETERINARIO)
- ativo
- ultimo_login
- data_criacao

2. VETERINARIOS

Armazena informações específicas dos profissionais veterinários.

Campos:

- id (PK)
- user_id (FK)
- crmv
- especialidade
- descricao

3. PETS

Representa os animais cadastrados pelos tutores.

Campos:

- id (PK)
- user_id (FK)
- nome
- especie

- raca
- data_nascimento
- observacoes

4. PET_VETERINARIO

Tabela de relacionamento entre pets e veterinários.

Campos:

- id (PK)
- pet_id (FK)
- veterinario_id (FK)
- data_inicio
- ativo
- observacoes

5. DIARIO_ENTRADAS

Representa cada “página diária” do diário do pet.

Campos:

- id (PK)
- pet_id (FK)
- data
- criado_em
- atualizado_em
- resumo
- humor_geral
- status

6. REGISTROS

Armazena informações detalhadas dentro de cada entrada do diário.

Campos:

- id (PK)
- entrada_id (FK)
- tipo (alimentação, atividade, comportamento etc.)
- subtipo
- valor

- unidade
- nota
- horario
- atualizado_em

7. COMENTARIOS

Comentários feitos por veterinários ou usuários sobre entradas do diário.

Campos:

- id (PK)
- entrada_id (FK)
- user_id (FK)
- comentario
- criado_em
- editado_em

8. INSIGHTS

Armazena análises geradas automaticamente pelo sistema (IA ou regras).

Campos:

- id (PK)
- pet_id (FK)
- tipo
- descricao
- origem_regra
- nivel_alerta (BAIXO, MEDIO, ALTO)
- data_inicio
- data_fim
- criado_em
- gerado_por
- status
- contexto

9. SOLICITACOES

Gerencia pedidos de vínculo entre tutor e veterinário.

Campos:

- id (PK)
- pet_id (FK)
- tutor_id (FK)
- veterinario_id (FK)
- status (PENDENTE, ACEITA, RECUSADA)
- mensagem
- criado_em
- respondido_em

10. LOGS_ERROS

Tabela de auditoria para registro de erros das procedures.

Campos:

- id (PK)
- nome_procedure
- usuario_bd
- data_erro
- codigo_erro
- mensagem_erro

Relacionamentos e Cardinalidade:

- USERS (1:N) — possui → PETS
- USERS (1:1) — possui → VETERINARIOS
- PETS (1:N) — possui → DIARIO_ENTRADAS
- DIARIO_ENTRADAS (1:N) — contém → REGISTROS
- DIARIO_ENTRADAS (1:N) — possui → COMENTARIOS
- PETS (N:N) — relação → VETERINARIOS (PET_VETERINARIO)
- PETS (1:N) — gera → INSIGHTS
- USERS (1:N) — realiza → SOLICITACOES

Esses relacionamentos garantem uma estrutura consistente, permitindo rastreabilidade completa do histórico de saúde dos pets e integração entre tutor e veterinário.

2.2 PROCEDURES E FUNCIONALIDADES IMPLEMENTADAS

Para atender os requisitos do sistema, foram criadas procedures responsáveis pela inserção de dados e controle de exceções.

Todas as procedures seguem o padrão:

- Recebimento de parâmetros (sem hard-code)
- Inserção na tabela correspondente
- Tratamento de exceções
- Registro de erros na tabela LOGS_ERROS

2.2.1 Procedure de Log de Erros

A procedure pr_log_erro é responsável por registrar todas as falhas ocorridas no sistema.

Ela grava:

- Nome da procedure
- Usuário do banco
- Data do erro
- Código do erro
- Mensagem de erro

2.2.2 Procedures de Inserção (Carga de Dados)

Foram criadas procedures para todas as tabelas principais:

- pr_insert_users → Cadastro de usuários
- pr_insert_veterinarios → Cadastro de veterinários
- pr_insert_pets → Cadastro de pets
- pr_insert_pet_veterinario → Relacionamento pet-vet
- pr_insert_diario_entradas → Criação de páginas do diário
- pr_insert_registros → Registros detalhados do dia
- pr_insert_comentarios → Comentários em entradas
- pr_insert_insights → Geração de insights automáticos
- pr_insert_solicitacoes → Solicitações de vínculo

2.2.3 Tratamento de Exceções

Cada procedure implementa tratamento de erros com:

- WHEN VALUE_ERROR
- WHEN NO_DATA_FOUND
- WHEN DUP_VAL_ON_INDEX (quando aplicável)
- WHEN OTHERS (obrigatório)

Em todos os casos, o erro é registrado na tabela LOGS_ERROS para auditoria e análise posterior.

2.2.4 Funcionalidades Implementadas

O sistema permite:

- Cadastro completo de usuários (tutores e veterinários)
- Registro de pets e histórico clínico
- Criação de diário diário estruturado
- Inserção de registros detalhados (alimentação, comportamento etc.)
- Comentários veterinários em entradas do diário
- Geração de insights automáticos baseados em regras
- Solicitação e gerenciamento de vínculo tutor-veterinário
- Rastreamento de erros via logs

2.2.5 Relatórios e Consultas (Blocos Anônimos)

Foram implementados blocos anônimos com:

- **Uso de JOINs**
- **GROUP BY**
- **ORDER BY**
- **Cálculos agregados**

Relatório 1 – Pets por tutor

Exibe quantidade de pets por tutor e espécie, com subtotal e total geral.

Relatório 2 – Insights por veterinário

Mostra quantidade de insights agrupados por veterinário e nível de alerta.

Relatório 3 – Análise de registros (LAG/LEAD)

Compara valores atuais, anteriores e posteriores dos registros de saúde.

Relatório 4 – Classificação de registros

Classifica registros em:

- BAIXO
- MÉDIO

- ALTO

com base no valor registrado.

3 SPRINT 3 – NOVAS FUNCIONALIDADES

3.1 Nova Entidade

3.1.1 Entidade ALERTAS

Na Sprint 3 foi adicionada a entidade ALERTAS, responsável pelo armazenamento de lembretes e alertas relacionados ao acompanhamento dos pets.

A tabela permite que um veterinário associe um alerta a determinado pet, definindo seu tipo, título, descrição, período de início e, quando aplicável, uma frequência de repetição.

Campos:

- id (PK)
- pet_id (FK)
- veterinario_id (FK)
- tipo
- titulo
- descricao
- data_inicio
- frequencia_horas
- data_fim
- ativo

O campo tipo possui valores controlados pelos seguintes tipos:

- CONSULTA
- REMEDIO
- ALIMENTACAO
- EXERCICIO
- OUTROS

Também foram implementadas regras de integridade para garantir que:

- a frequência, quando informada, seja maior que zero;
- o campo ativo aceite apenas os valores 0 ou 1;
- a data_fim, quando informada, não seja anterior à data_inicio.

A estrutura foi implementada diretamente no banco por meio de restrições CHECK e chaves estrangeiras.

3.1.2 Relacionamento de ALERTAS com PETS e VETERINARIOS

A entidade ALERTAS possui relacionamentos com PETS e VETERINARIOS.

Cada alerta está obrigatoriamente associado a um pet por meio da chave estrangeira pet_id e a um veterinário por meio da chave estrangeira veterinario_id.

Dessa forma, os relacionamentos podem ser representados como:

- PETS (1:N) → ALERTAS
- VETERINARIOS (1:N) → ALERTAS

Essa estrutura permite identificar tanto o animal ao qual o alerta pertence quanto o profissional responsável pelo seu acompanhamento.

3.2 Funções Implementadas

Na Sprint 3 foram desenvolvidas duas funções em PL/SQL com finalidades relacionadas ao domínio do projeto. A primeira realiza uma transformação de dados relacionais para uma representação JSON construída manualmente, enquanto a segunda realiza o cálculo da idade de um pet a partir de sua data de nascimento.

3.2.1 Função 1 – Transformação Relacional para JSON

Foi criada a função fn_pet_json, que recebe como parâmetro o identificador de um pet e retorna uma representação de seus dados no formato JSON.

A função realiza uma consulta envolvendo os dados do pet e de seu tutor e também identifica o veterinário responsável pelo acompanhamento ativo.

As informações retornadas incluem:

- ID do pet;
- nome;
- espécie;
- raça;
- data de nascimento;
- tutor;
- veterinário.

A transformação para JSON é realizada **manualmente por concatenação de strings**, sem utilização de funções nativas de conversão JSON como JSON_OBJECT, JSON_VALUE, JSON_QUERY ou JSON_TABLE.

A função também possui tratamento de exceções. Caso o pet não seja encontrado, é retornada uma mensagem de erro em formato JSON. Também são tratados erros de conversão e erros inesperados.

Funcionamento

O processo pode ser resumido da seguinte maneira:

ID do PET

↓

Consulta na tabela PETS

↓

JOIN com USERS

↓

Busca do veterinário responsável

↓

Obtenção dos dados relacionais

↓

Concatenação manual

↓

JSON do PET

3.2.2 Função 2 – Cálculo da Idade do Pet

Foi criada a função fn_calcular_idade_pet, responsável por calcular a idade de um pet utilizando sua data de nascimento armazenada na tabela PETS.

A função recebe o id do pet como parâmetro, consulta sua data de nascimento e calcula a idade utilizando a diferença entre a data atual e a data de nascimento.

O cálculo é realizado por meio da função MONTHS_BETWEEN, considerando a quantidade de meses decorridos e convertendo o resultado para anos inteiros.

A função também possui tratamentos para:

- pet inexistente;

- data de nascimento futura;
- data de nascimento nula ou inválida;
- outros erros inesperados.

Quando ocorre uma dessas situações, a função informa o erro por meio do DBMS_OUTPUT e retorna NULL.

Essa abordagem evita a necessidade de armazenar uma coluna de idade na tabela, pois a idade pode ser calculada dinamicamente a partir da data de nascimento.

3.3 Procedures Implementadas

3.3.1 Procedure 1 – Relatório de Pets em JSON

Foi desenvolvida a procedure pr_relatorio_pets_json, responsável por realizar um relatório envolvendo informações de pets, tutores e veterinários e apresentar os resultados juntamente com a representação JSON produzida pela função fn_pet_json.

A procedure utiliza JOIN entre as tabelas:

- PETS;
- USERS;
- PET_VETERINARIO;
- VETERINARIOS.

Além disso, utiliza LEFT JOIN para permitir que um pet seja apresentado mesmo quando não houver um veterinário ativo associado.

Para cada registro encontrado, são exibidas duas informações:

1. os dados obtidos pelo JOIN;
2. o JSON correspondente ao pet.

A função fn_pet_json é reutilizada pela procedure, demonstrando a integração entre as funcionalidades implementadas na Sprint 3.

A procedure também possui tratamento de exceções para ausência de registros, valores inválidos e erros inesperados, utilizando a procedure pr_log_erro já existente no projeto.

3.3.2 Procedure 2 – Agrupamento Manual de Registros

A segunda procedure desenvolvida foi a pr_relatorio_registros_agrupado.

Ela utiliza a tabela REGISTROS como fonte dos dados e realiza um agrupamento manual utilizando:

- TIPO;
- SUBTIPO;
- VALOR.

O objetivo é apresentar:

- os valores detalhados por combinação de TIPO e SUBTIPO;
- o subtotal de cada TIPO;
- o total geral dos registros.

Um requisito importante dessa implementação é que o agrupamento e as somas sejam realizados **manualmente**, sem utilização de SUM, ROLLUP, CUBE, GROUPING SETS ou GROUPING.

Para isso, a procedure utiliza um cursor e variáveis de controle para identificar mudanças de categoria e calcular progressivamente os valores.

O processamento segue a lógica:

REGISTROS

↓

Ordenação por TIPO e SUBTIPO

↓

Leitura através de cursor

↓

Soma do SUBTIPO

↓

Soma do TIPO

↓

Soma do TOTAL GERAL

↓

Exibição dos resultados

A procedure também verifica se existem registros numéricos e possui tratamento de exceções para ausência de dados, valores inválidos e erros inesperados.

Carga complementar para o relatório

Para garantir uma quantidade suficiente de registros numéricos para a demonstração, foram adicionados dois registros complementares à tabela REGISTROS.

Essa carga permitiu atingir pelo menos cinco registros numéricos e também demonstrar a soma de registros pertencentes à mesma combinação de categorias.

3.4 Auditoria e Trigger

3.4.1 Tabela de Auditoria

Para complementar o controle das operações realizadas no banco, foi criada a tabela AUDITORIA_REGISTROS.

Essa tabela tem como objetivo armazenar o histórico das operações realizadas sobre a tabela REGISTROS.

São armazenadas informações como:

- usuário do banco;
- tipo da operação;
- data e hora da operação;
- valores anteriores;
- valores novos.

A estrutura possui campos específicos para armazenar os valores OLD e NEW, permitindo identificar exatamente o estado do registro antes e depois de uma alteração.

As informações armazenadas variam conforme a operação:

Operação	Informações registradas
INSERT	valores NEW
UPDATE	valores OLD e NEW

DELETE valores OLD

3.4.2 Trigger de Auditoria DML

Foi implementada a trigger trg_auditoria_registros, associada à tabela REGISTROS.

A trigger é executada **após operações de INSERT, UPDATE ou DELETE**, permitindo registrar automaticamente as alterações realizadas na tabela de auditoria.

A implementação utiliza as condições:

- INSERTING;
- UPDATING;
- DELETING.

Para cada situação, são inseridos os dados correspondentes na tabela AUDITORIA_REGISTROS.

No caso de um INSERT, são armazenados os valores novos utilizando :NEW.

No caso de um UPDATE, são armazenados tanto os valores anteriores (:OLD) quanto os valores posteriores (:NEW).

No caso de um DELETE, são armazenados os valores anteriores utilizando :OLD.

Com isso, a trigger fornece um mecanismo automático de auditoria das operações DML realizadas sobre os registros.

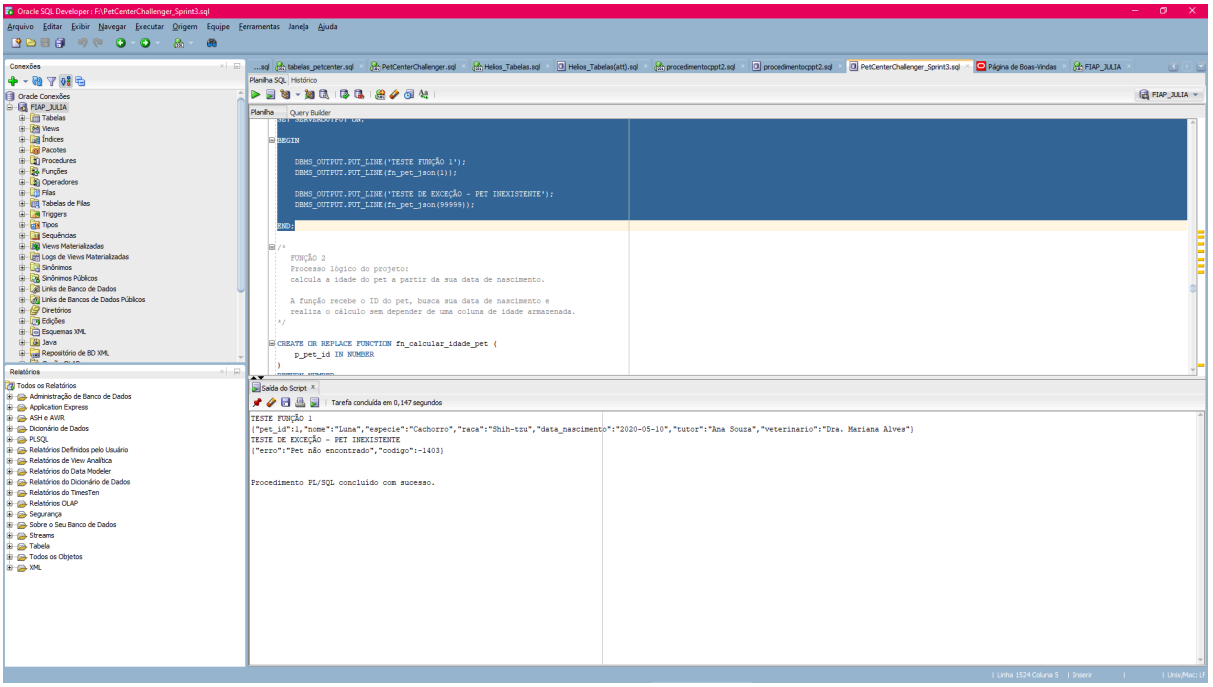
3.5 Testes e Evidências de Execução

3.5.1 Evidência da Função 1 – JSON

Foi realizado um teste da função fn_pet_json utilizando um pet existente e, posteriormente, um identificador inexistente.

O primeiro teste demonstra a geração correta do JSON contendo os dados relacionais do pet.

O segundo teste demonstra o tratamento da exceção para um pet que não existe no banco.



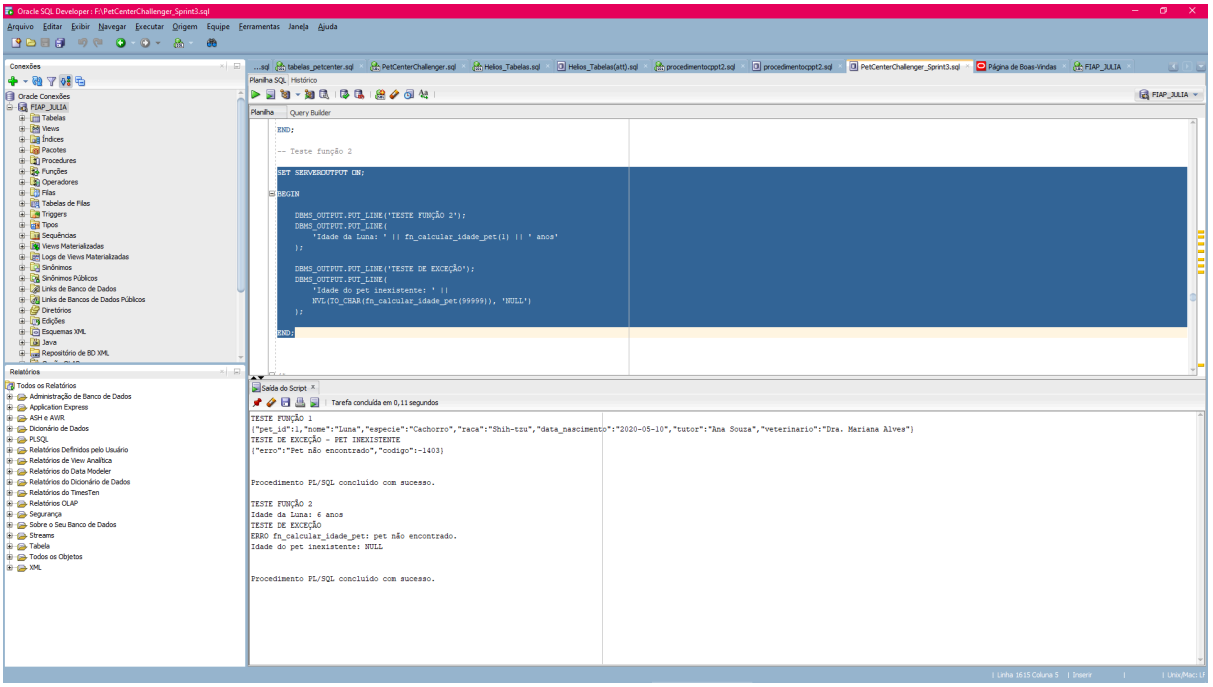
O teste demonstra a transformação dos dados relacionais do pet em uma estrutura JSON e o tratamento de erro para um identificador inexistente.

3.5.2 Evidência da Função 2 – Cálculo da Idade

Foi realizado um teste utilizando o pet Luna e, posteriormente, um identificador inexistente.

A primeira execução demonstra o cálculo da idade a partir da data de nascimento armazenada na tabela PETS.

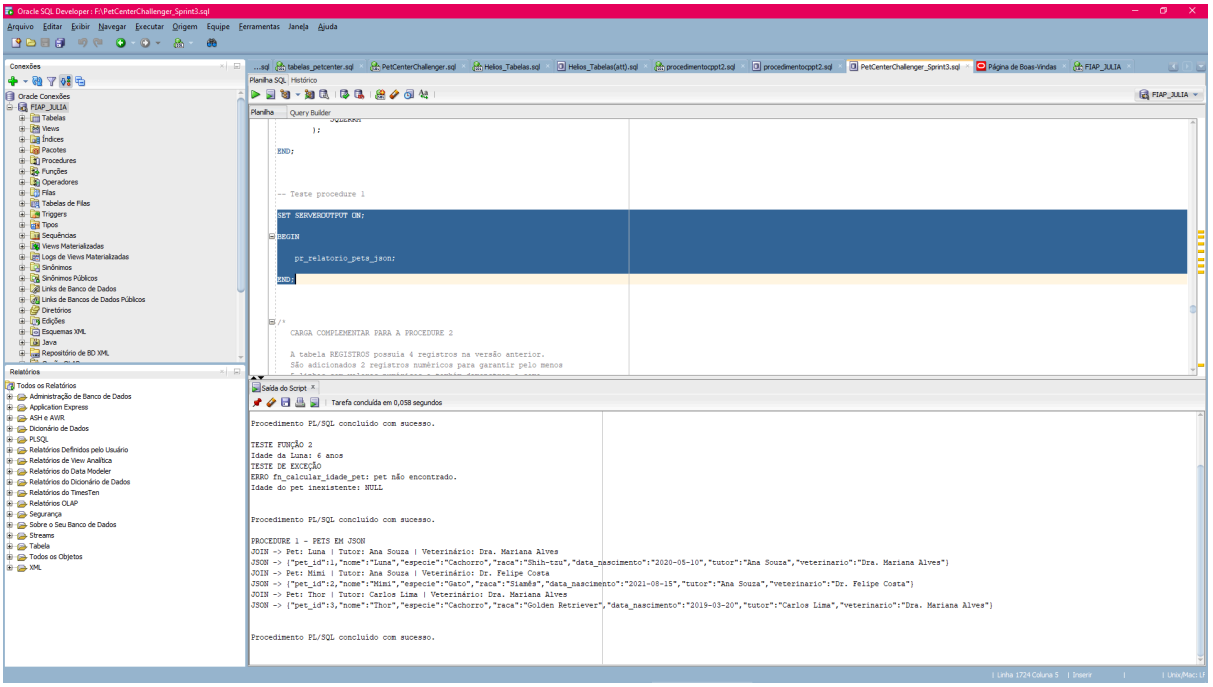
A segunda execução demonstra o tratamento da situação em que o pet informado não existe.



O resultado demonstra o cálculo dinâmico da idade do pet e o tratamento da exceção para um pet inexistente.

3.5.3 Evidência da Procedure 1 – Pets em JSON

Foi executada a procedure pr_relatorio_pets_json, responsável por realizar os JOINs entre as entidades relacionadas e apresentar os dados juntamente com a representação JSON de cada pet.



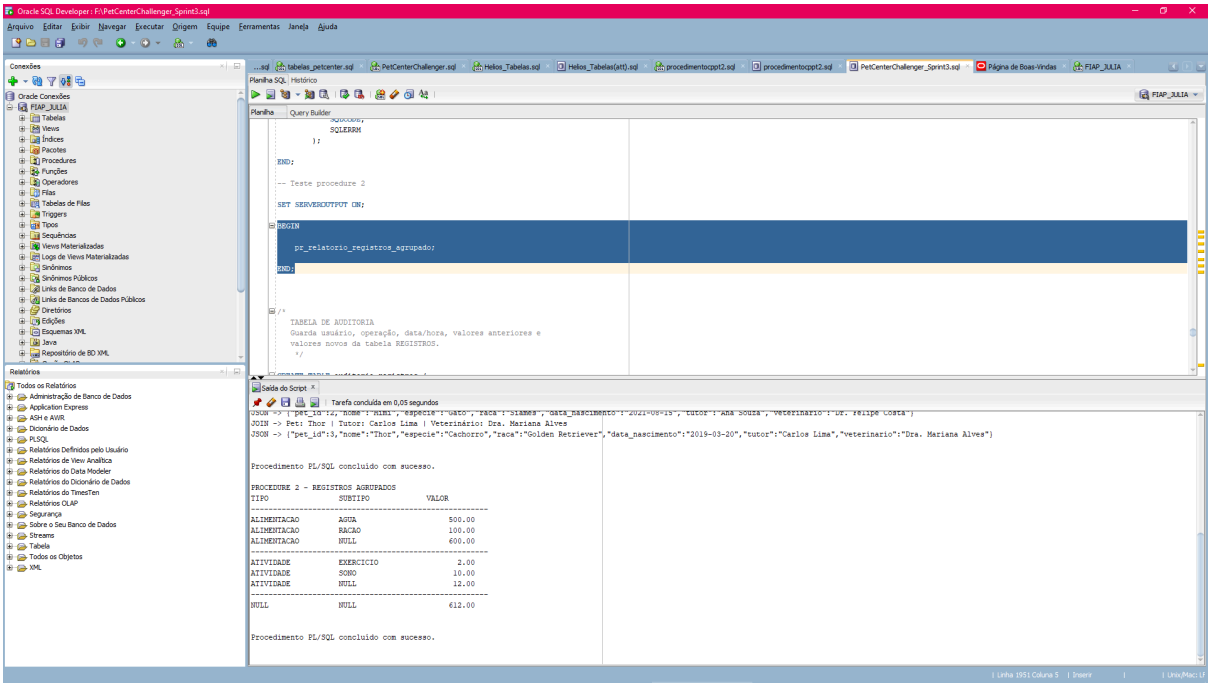
A evidência demonstra a execução da procedure, os dados obtidos por meio dos JOINs e a geração do JSON utilizando a função fn_pet_json.

3.5.4 Evidência da Procedure 2 – Registros Agrupados

Foi executada a procedure pr_relatorio_registros_agrupado, demonstrando o agrupamento manual dos registros.

A saída apresenta:

- combinações de tipo e subtipo;
- somas por combinação;
- subtotais por tipo;
- total geral.



A evidência demonstra o agrupamento manual dos registros e o cálculo dos subtotais e do total geral sem utilização das funções de agregação e recursos de agrupamento proibidos pelo requisito.

3.5.5 Evidência da Trigger de Auditoria

Print 1 – Execução dos comandos DML e Registros gerados na auditoria

Foi realizado um teste da trigger utilizando três operações sobre a tabela REGISTROS:

1. INSERT;
2. UPDATE;
3. DELETE.

O teste utiliza um registro criado especificamente para a validação da trigger. Após as três operações, é realizada a confirmação da execução.

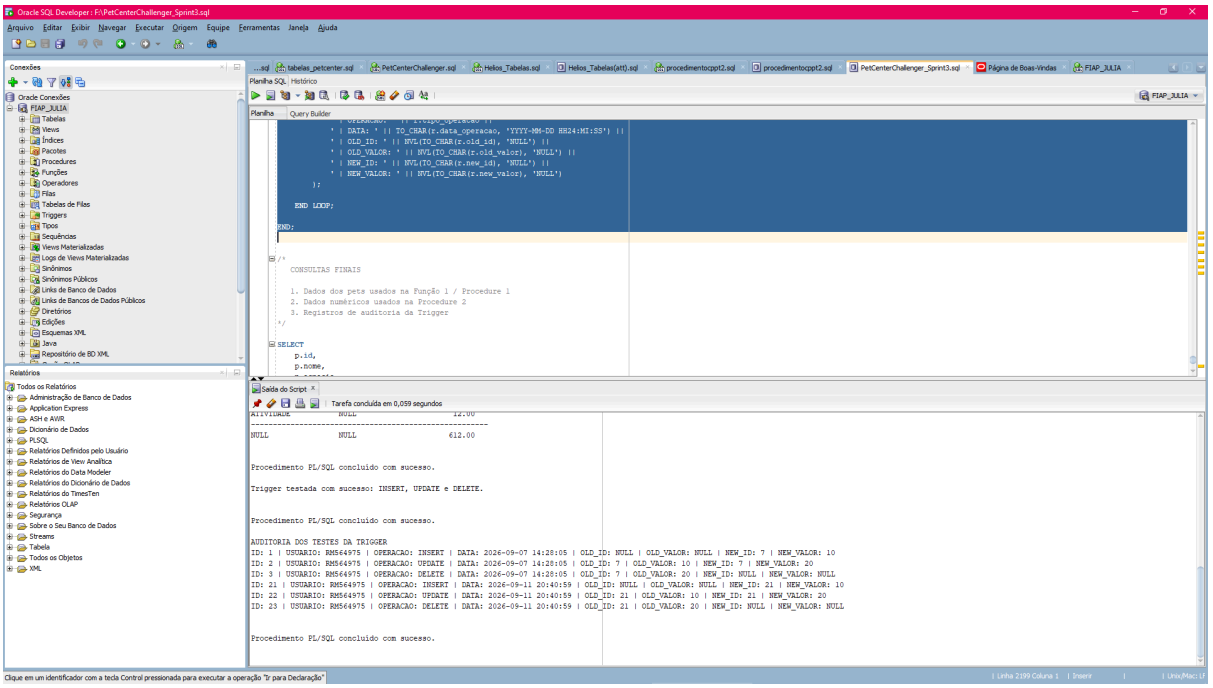
O teste demonstra a execução das operações INSERT, UPDATE e DELETE utilizadas para validar o funcionamento da trigger.

Após a execução das operações DML, foi realizada uma consulta na tabela AUDITORIA_REGISTROS para verificar os registros criados automaticamente pela trigger.

A consulta apresenta:

- identificador da auditoria;

- usuário;
- operação;
- data;
- valores antigos;
- valores novos.



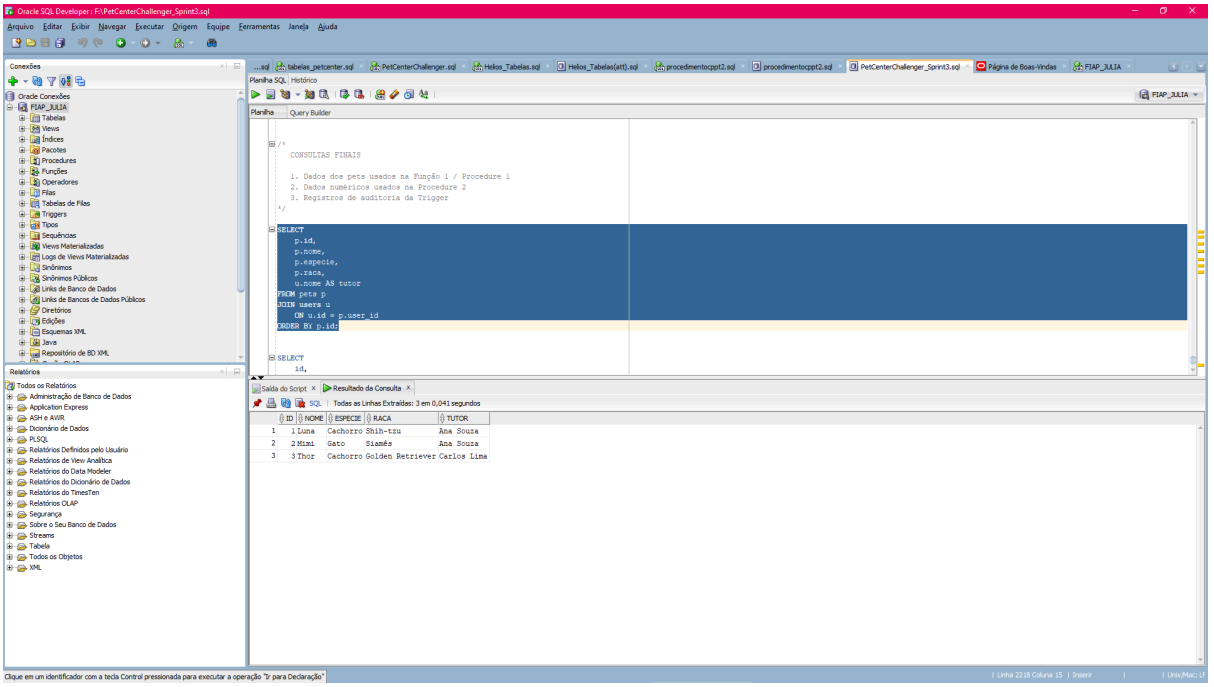
A evidência comprova que as operações realizadas na tabela REGISTROS foram registradas automaticamente na tabela AUDITORIA_REGISTROS, incluindo os valores anteriores e posteriores quando aplicável.

3.5.6 Consultas Finais de Validação

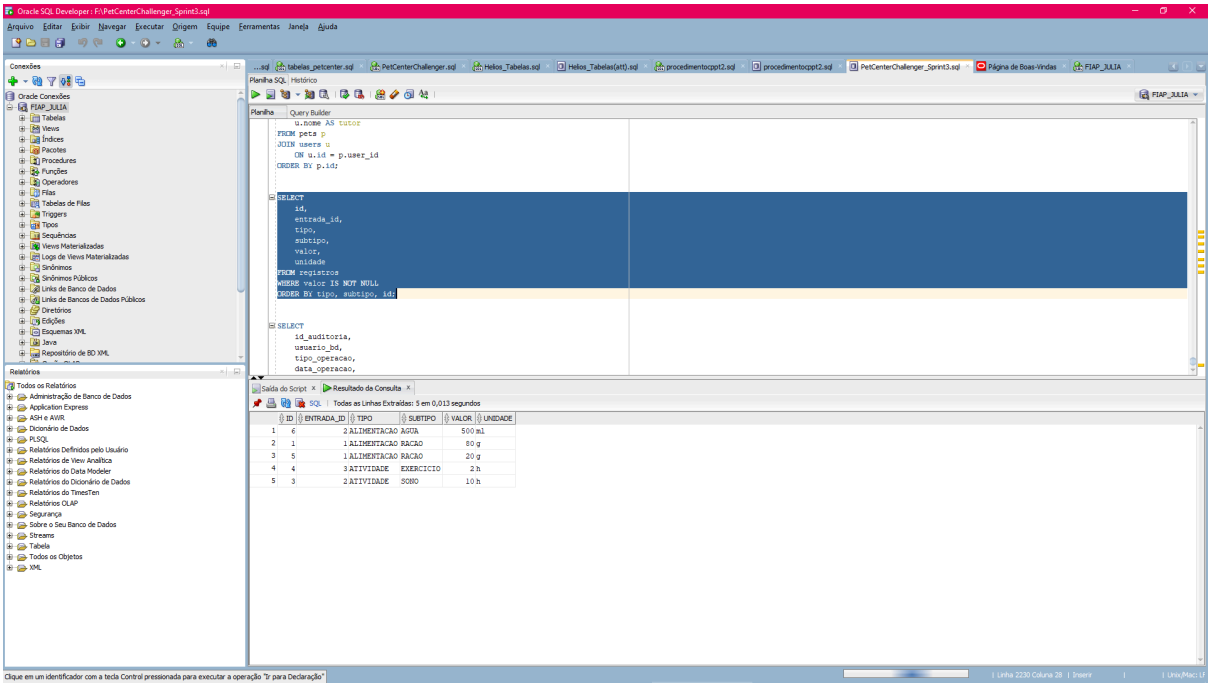
Por fim, foram realizadas consultas de validação para verificar os dados utilizados nas funcionalidades da Sprint 3.

Foram realizadas consultas referentes a:

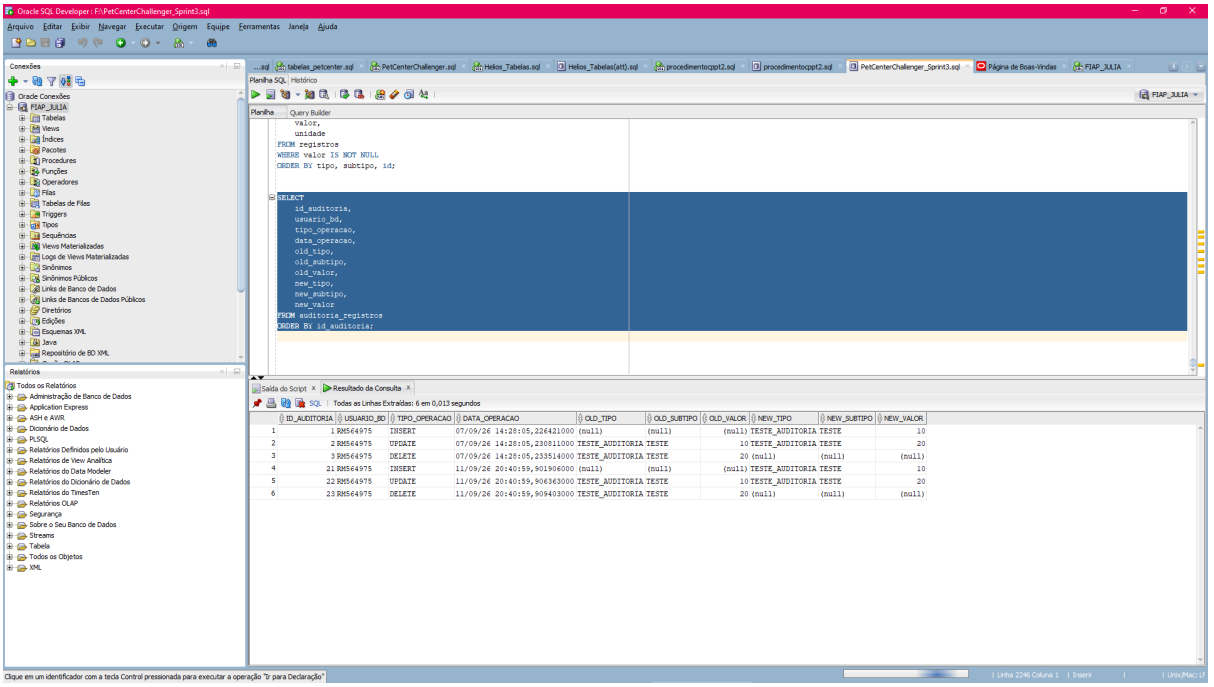
- dados dos pets utilizados pela Função 1 e Procedure 1;



- registros numéricos utilizados pela Procedure 2;



- registros gerados na tabela de auditoria pela trigger.



Essas consultas permitem validar diretamente os dados armazenados nas tabelas após a execução das funcionalidades.

4 - SPRINT 4: EMPACOTAMENTO

4.1 Objetivo

Reorganizar os objetos do Oracle (procedures, funções e lógica de auditoria) em packages, seguindo modularidade e reutilização, e deixar o script rerrodável do zero (limpeza + criação + carga + testes).

4.2 Organização dos packages

Package	Conteúdo
pkg_petcenter_carga	pr_log_erro e todas as pr_insert_* (users, veterinarios, pets, pet_veterinario, diario_entradas, registros, comentarios, insights, solicitacoes, alertas)

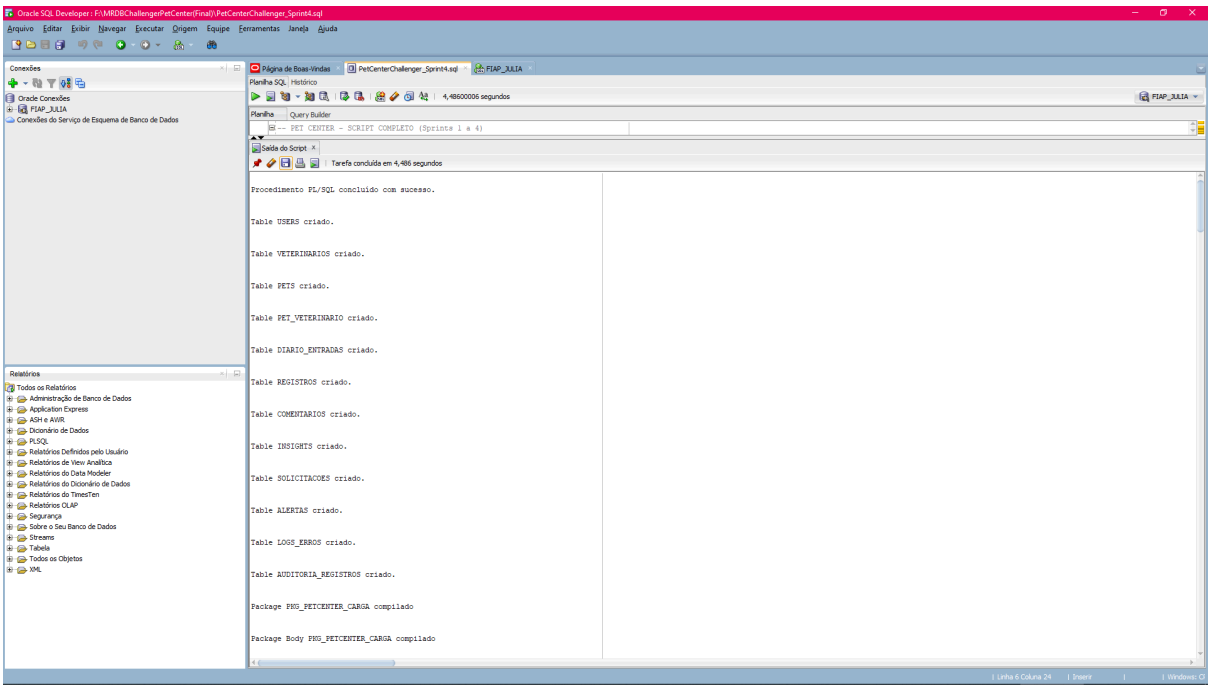
pkg_petcenter_relatorios	fn_pet_json, fn_calcular_idade_pet, pr_relatorio_pets_json, pr_relatorio_registros_agrupado, fn_dataset_pets_json (export JSON aninhado para MongoDB)
pkg_petcenter_auditoria	pr_auditar_registros (INSERT/UPDATE/DELETE + atualizado_em)

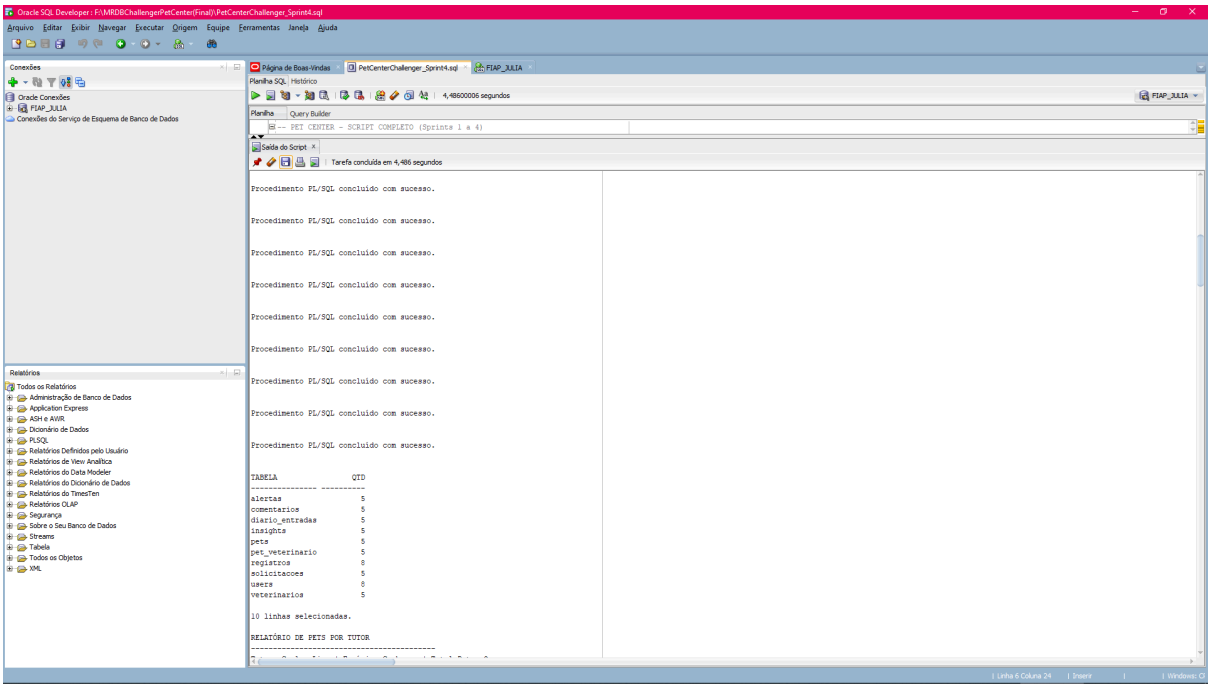
Trigger: trg_auditoria_registros permanece fora do package (limitação do Oracle). Ela ficou fina e só chama pkg_petcenter_auditoria.pr_auditar_registros.

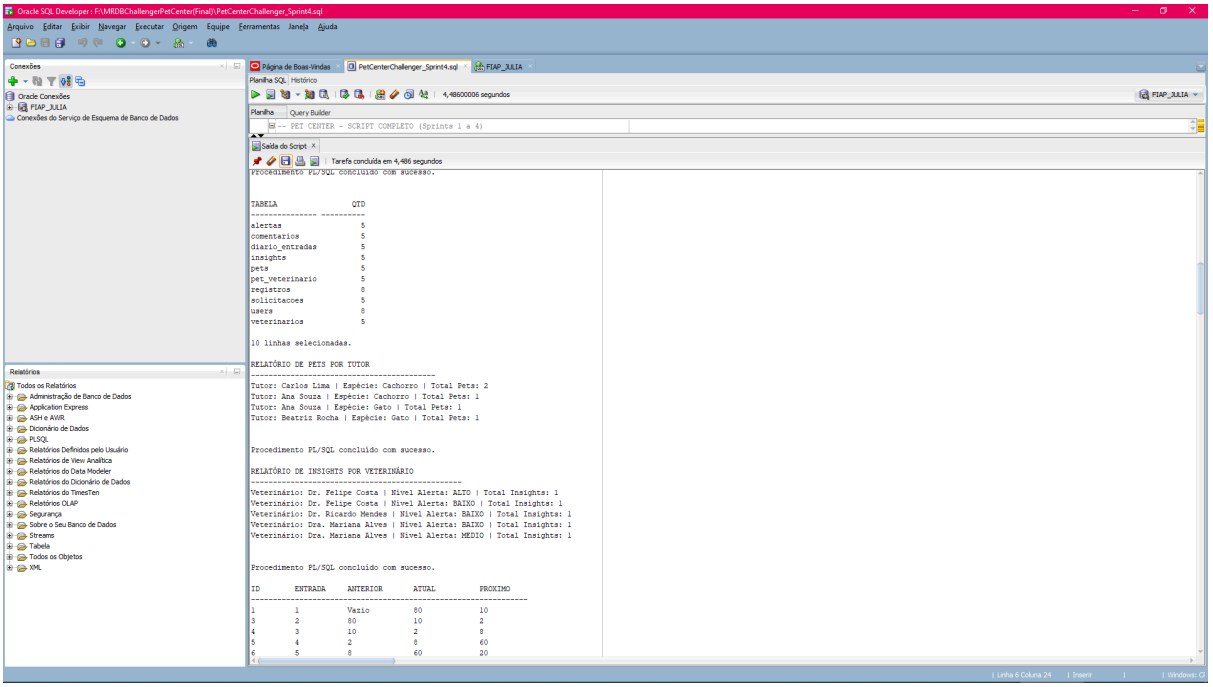
4.3 Refatoração do script SQL

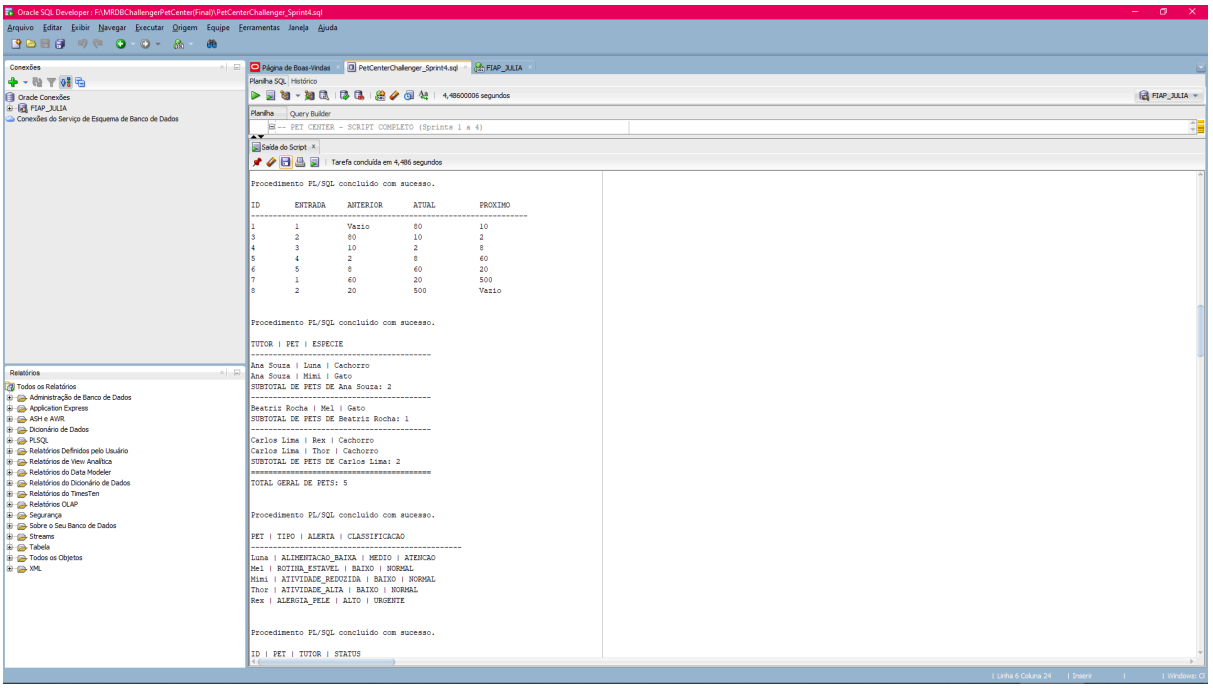
1. Seção 0 – Limpeza: drop de trigger, packages, procedures/funções avulsas e tabelas (CASCADE CONSTRAINTS), para o professor executar o arquivo inteiro (F5) sem ALTER TABLE.
2. Tabelas: auditoria_registros já nasce com old_atualizado_em e new_atualizado_em.
3. Carga única: removida a “carga complementar” espalhada; todas as tabelas de negócio com ≥ 5 registros em um só bloco.
4. Chamadas atualizadas: inserts e testes usam pkg_....procedimento (ex.: pkg_petcenter_carga.pr_insert_pets).
5. Terminadores /: após packages, trigger e blocos PL/SQL, para compilação correta no SQL Developer.
6. Evidências de exceção: bloco que força erro em cada procedure e grava em logs_erros.
7. Export JSON: fn_dataset_pets_json gera array de pets com tutor, veterinário, diário, registros, comentários, insights e alertas (documento NoSQL).

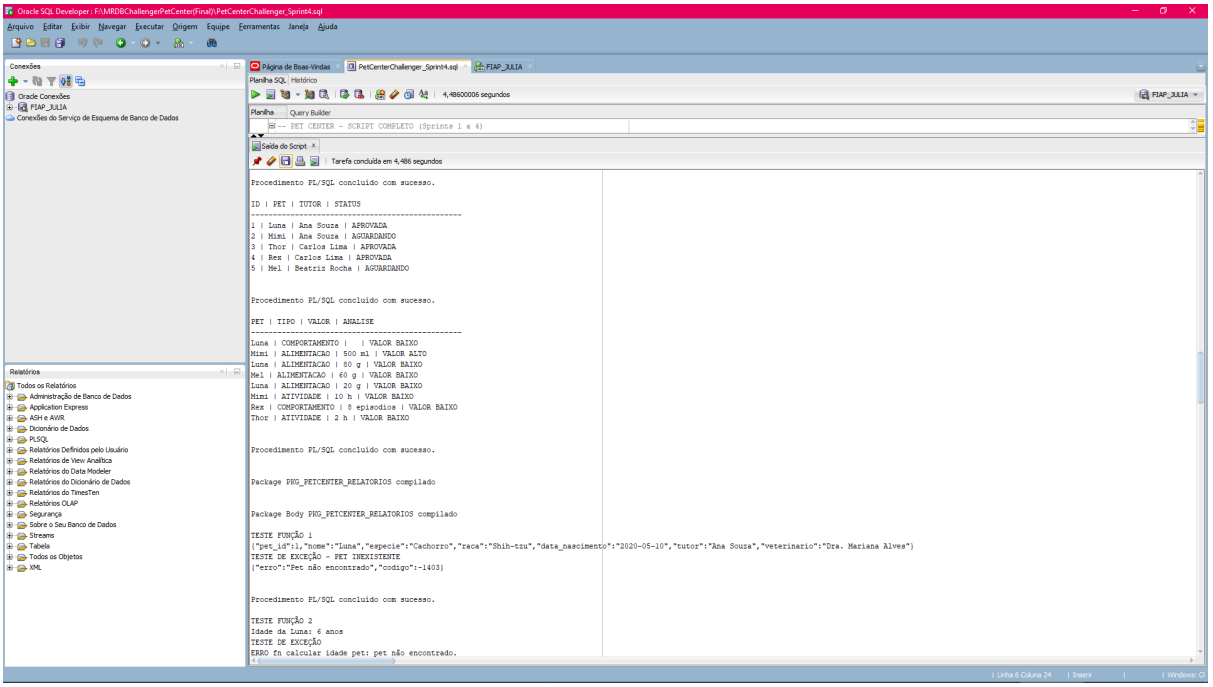
4.4 Evidências

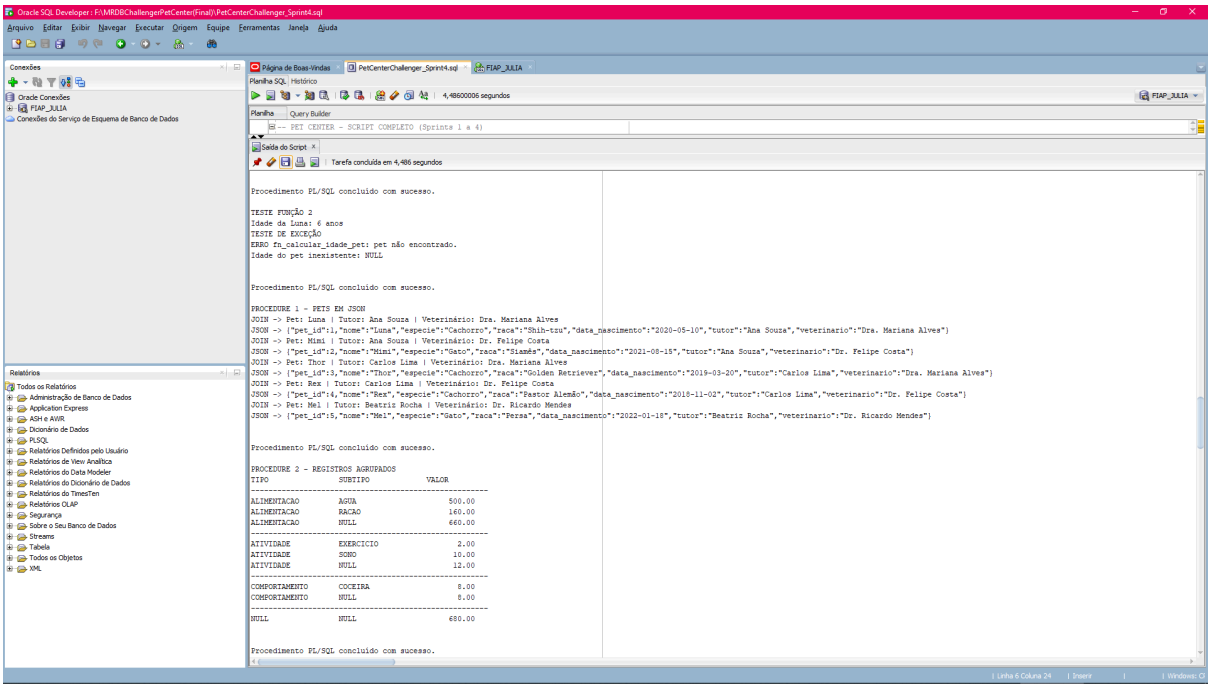

Oracle SQL Developer : F:\MRDBChallengerPetCenter(Final)\PetCenterChallenger_Sprint4.sql
Arquivo Editar Exibir Navegar Executar Origem Equipe Ferramentas Janela Ajuda
Conexões
Oracle Conexões
FIAP_JULIA
Conexões do Serviço de Esquema de Banco de Dados
Relatórios
Todos os Relatórios
Administração de Banco de Dados
Application Express
ASH e AWR
Dicionário de Dados
PLSQL
Relatórios Definidos pelo Usuário
Relatórios de View Analítica
Relatórios do Data Modeler
Relatórios do Dicionário de Dados
Relatórios do TimesTen
Relatórios OLAP
Segurança
Sobre o Seu Banco de Dados
Streams
Tabela
Todos os Objetos
XML
Página de Boas-Vindas
PetCenterChallenger_Sprint4.sql
FIAP_JULIA
Planilha SQL Histórico
4,48600006 segundos
Planilha Query Builder
-- PET CENTER - SCRIPT COMPLETO (Sprints 1 a 4)
Saída do Script
Tarefa concluída em 4,486 segundos
Procedimento PL/SQL concluído com sucesso.
Table USERS criado.
Table VETERINARIOS criado.
Table PETS criado.
Table PET_VETERINARIO criado.
Table DIARIO_ENTRADAS criado.
Table REGISTROS criado.
Table COMENTARIOS criado.
Table INSIGHTS criado.
Table SOLICITACOES criado.
Table ALERTAS criado.
Table LOGS_ERROS criado.
Table AUDITORIA_REGISTROS criado.
Package PKG_PETCENTER_CARGA compilado
Package Body PKG_PETCENTER_CARGA compilado
Linha 6 Coluna 24
Inserir
Windows:

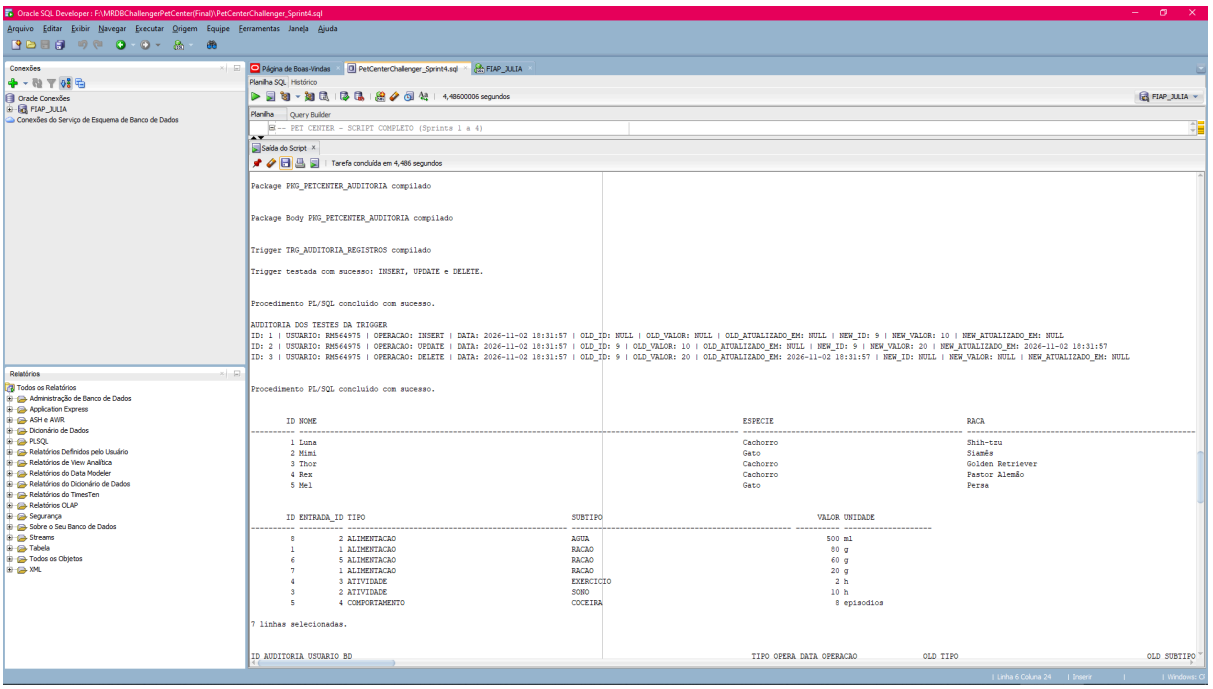

Oracle SQL Developer : F:\MRDBChallengerPetCenter(Final)\PetCenterChallenger_Sprint4.sql
Arquivo Editar Exibir Navegar Executar Origem Equipe Ferramentas Janela Ajuda
Conexões
Oracle Conexões
FIAP_JULIA
Conexões do Serviço de Esquema de Banco de Dados
Relatórios
Todos os Relatórios
Administração de Banco de Dados
Application Express
ASH e AWR
Dicionário de Dados
PLSQL
Relatórios Definidos pelo Usuário
Relatórios de View Analítica
Relatórios do Data Modeler
Relatórios do Dicionário de Dados
Relatórios do TimesTen
Relatórios OLAP
Segurança
Sobre o Seu Banco de Dados
Streams
Tabela
Todos os Objetos
XML
Página de Boas-Vindas
PetCenterChallenger_Sprint4.sql
FIAP_JULIA
Planilha SQL Histórico
4,48600006 segundos
Planilha Query Builder
-- PET CENTER - SCRIPT COMPLETO (Sprints 1 a 4)
Saída do Script
Tarefa concluída em 4,486 segundos
Procedimento PL/SQL concluído com sucesso.
Procedimento PL/SQL concluído com sucesso.
Procedimento PL/SQL concluído com sucesso.
Procedimento PL/SQL concluído com sucesso.
Procedimento PL/SQL concluído com sucesso.
Procedimento PL/SQL concluído com sucesso.
Procedimento PL/SQL concluído com sucesso.
Procedimento PL/SQL concluído com sucesso.
Procedimento PL/SQL concluído com sucesso.
TABELA QTD
alertas 5
comentarios 5
diario_entradas 5
insights 5
pets 5
pet_veterinario 5
registros 8
solicitacoes 5
users 6
veterinarios 5
10 linhas selecionadas.
RELATÓRIO DE PETS POR TUTOR
Linha 6 Coluna 24
Inserir
Windows:

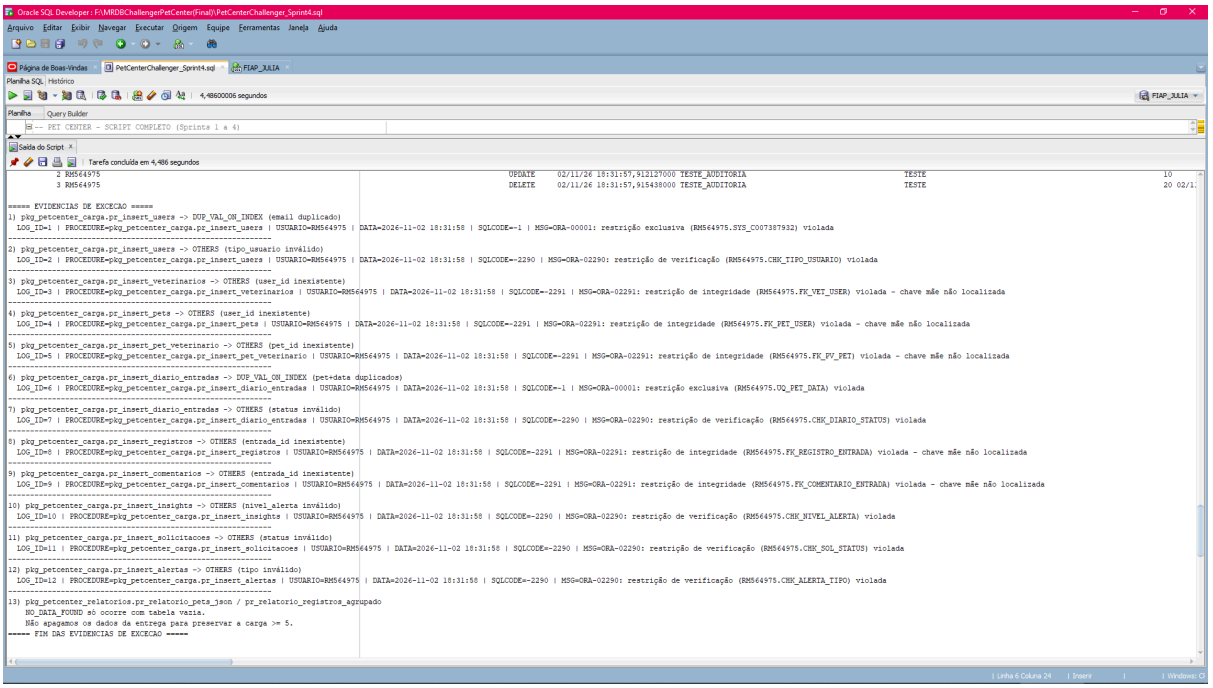

Oracle SQL Developer : F:\MRDBChallengerPetCenter(Final)\PetCenterChallenger_Sprint4.sql
Arquivo Editar Exibir Navegar Executar Origem Equipe Ferramentas Janela Ajuda
Conexões
Oracle Conexões
FIAP_JULIA
Conexões do Serviço de Esquema de Banco de Dados
Relatórios
Todos os Relatórios
Administração de Banco de Dados
Application Express
ASH e AWR
Dicionário de Dados
PLSQL
Relatórios Definidos pelo Usuário
Relatórios de View Analítica
Relatórios do Data Modeler
Relatórios do Dicionário de Dados
Relatórios do TimesTen
Relatórios OLAP
Segurança
Sobre o Seu Banco de Dados
Streams
Tabela
Todos os Objetos
XML
Página de Boas-Vindas
PetCenterChallenger_Sprint4.sql
FIAP_JULIA
Planilha SQL Histórico
4,48600006 segundos
Planilha Query Builder
-- PET CENTER - SCRIPT COMPLETO (Sprints 1 a 4)
Saída do Script
Tarefa concluída em 4,486 segundos
Procedimento PL/SQL concluído com sucesso.
TABELA QTD
alertas 5
comentarios 5
diario_entradas 5
insights 5
pets 5
pet_veterinario 5
registros 8
solicitacoes 5
users 8
veterinarios 5
10 linhas selecionadas.
RELATÓRIO DE PETS POR TUTOR
Tutor: Carlos Lima | Espécie: Cachorro | Total Pets: 2
Tutor: Ana Souza | Espécie: Cachorro | Total Pets: 1
Tutor: Ana Souza | Espécie: Gato | Total Pets: 1
Tutor: Beatriz Rocha | Espécie: Gato | Total Pets: 1
Procedimento PL/SQL concluído com sucesso.
RELATÓRIO DE INSIGHTS POR VETERINÁRIO
Veterinário: Dr. Felipe Costa | Nível Alerta: ALTO | Total Insights: 1
Veterinário: Dr. Felipe Costa | Nível Alerta: BAIXO | Total Insights: 1
Veterinário: Dr. Ricardo Mendes | Nível Alerta: BAIXO | Total Insights: 1
Veterinário: Dra. Mariana Alves | Nível Alerta: BAIXO | Total Insights: 1
Veterinário: Dra. Mariana Alves | Nível Alerta: MEDIO | Total Insights: 1
Procedimento PL/SQL concluído com sucesso.
ID ENTRADA ANTERIOR ATUAL PROXIMO
1 1 Vazio 80 10
3 2 80 10 2
4 3 10 2 8
5 4 2 8 60
6 5 8 60 20
Linha 6 Coluna 24
Inserir
Windows:

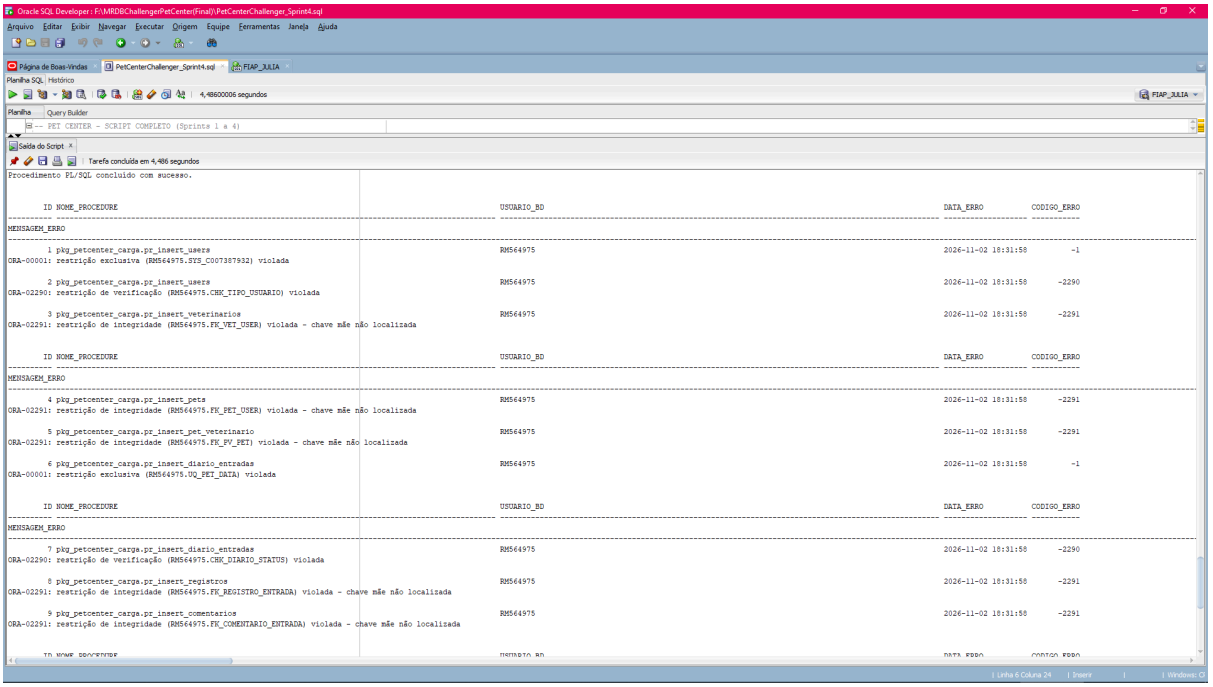

Oracle SQL Developer : F:\MRDBChallengerPetCenter(Final)\PetCenterChallenger_Sprint4.sql
Arquivo Editar Exibir Navegar Executar Origem Equipe Ferramentas Janela Ajuda
Conexões
Oracle Conexões
FIAP_JULIA
Conexões do Serviço de Esquema de Banco de Dados
Relatórios
Todos os Relatórios
Administração de Banco de Dados
Application Express
ASH e AWR
Dicionário de Dados
PLSQL
Relatórios Definidos pelo Usuário
Relatórios de View Analítica
Relatórios do Data Modeler
Relatórios do Dicionário de Dados
Relatórios do TimesTen
Relatórios OLAP
Segurança
Sobre o Seu Banco de Dados
Streams
Tabela
Todos os Objetos
XML
Página de Boas-Vindas
PetCenterChallenger_Sprint4.sql
FIAP_JULIA
Planilha SQL Histórico
4,48600006 segundos
Planilha Query Builder
-- PET CENTER - SCRIPT COMPLETO (Sprints 1 a 4)
Saída do Script
Tarefa concluída em 4,486 segundos
Procedimento PL/SQL concluído com sucesso.
ID ENTRADA ANTERIOR ATUAL PROXIMO
1 1 Vazio 80 10
3 2 80 10 2
4 3 10 2 8
5 4 2 8 60
6 5 8 60 20
7 1 60 20 500
8 2 20 500 Vazio
Procedimento PL/SQL concluído com sucesso.
TUTOR | PET | ESPECIE
Ana Souza | Luna | Cachorro
Ana Souza | Mimi | Gato
SUBTOTAL DE PETS DE Ana Souza: 2
Beatriz Rocha | Mel | Gato
SUBTOTAL DE PETS DE Beatriz Rocha: 1
Carlos Lima | Rex | Cachorro
Carlos Lima | Thor | Cachorro
SUBTOTAL DE PETS DE Carlos Lima: 2
TOTAL GERAL DE PETS: 5
Procedimento PL/SQL concluído com sucesso.
PET | TIPO | ALERTA | CLASSIFICACAO
Luna | ALIMENTACAO_BAIXA | MEDIO | ATENCAO
Mel | ROTINA_ESTAVEL | BAIXO | NORMAL
Mimi | ATIVIDADE_REDUZIDA | BAIXO | NORMAL
Thor | ATIVIDADE_ALTA | BAIXO | NORMAL
Rex | ALERGIA_PELE | ALTO | URGENTE
Procedimento PL/SQL concluído com sucesso.
ID | PET | TUTOR | STATUS
Linha 6 Coluna 24
Inserir
Windows:


Oracle SQL Developer : F:\MRDBChallengerPetCenter(Final)\PetCenterChallenger_Sprint4.sql
Arquivo Editar Exibir Navegar Executar Origem Equipe Ferramentas Janela Ajuda
Conexões
Oracle Conexões
FIAP_JULIA
Conexões do Serviço de Esquema de Banco de Dados
Relatórios
Todos os Relatórios
Administração de Banco de Dados
Application Express
ASH e AWR
Dicionário de Dados
PLSQL
Relatórios Definidos pelo Usuário
Relatórios de View Analítica
Relatórios do Data Modeler
Relatórios do Dicionário de Dados
Relatórios do TimesTen
Relatórios OLAP
Segurança
Sobre o Seu Banco de Dados
Streams
Tabela
Todos os Objetos
XML
Página de Boas-Vindas
PetCenterChallenger_Sprint4.sql
FIAP_JULIA
Planilha SQL
Histórico
4,48600006 segundos
Planilha
Query Builder
-- PET CENTER – SCRIPT COMPLETO (Sprints 1 a 4)
Saída do Script
Tarefa concluída em 4,486 segundos
Procedimento PL/SQL concluído com sucesso.
ID | PET | TUTOR | STATUS
1 | Luna | Ana Souza | APROVADA
2 | Mimi | Ana Souza | AGUARDANDO
3 | Thor | Carlos Lima | APROVADA
4 | Rex | Carlos Lima | APROVADA
5 | Mel | Beatriz Rocha | AGUARDANDO
Procedimento PL/SQL concluído com sucesso.
PET | TIPO | VALOR | ANALISE
Luna | COMPORTAMENTO | | VALOR BAIXO
Mimi | ALIMENTACAO | 500 ml | VALOR ALTO
Luna | ALIMENTACAO | 80 g | VALOR BAIXO
Mel | ALIMENTACAO | 60 g | VALOR BAIXO
Luna | ALIMENTACAO | 20 g | VALOR BAIXO
Mimi | ATIVIDADE | 10 h | VALOR BAIXO
Rex | COMPORTAMENTO | 8 episodios | VALOR BAIXO
Thor | ATIVIDADE | 2 h | VALOR BAIXO
Procedimento PL/SQL concluído com sucesso.
Package PKG_PETCENTER_RELATORIOS compilado
Package Body PKG_PETCENTER_RELATORIOS compilado
TESTE FUNÇÃO 1
{"pet_id":1,"nome":"Luna","especie":"Cachorro","raca":"Shih-tzu","data_nascimento":"2020-05-10","tutor":"Ana Souza","veterinario":"Dra. Mariana Alves"}
TESTE DE EXCEÇÃO - PET INEXISTENTE
{"erro":"Pet não encontrado","codigo":-1403}
Procedimento PL/SQL concluído com sucesso.
TESTE FUNÇÃO 2
Idade da Luna: 6 anos
TESTE DE EXCEÇÃO
ERRO fn_calcular_idade_pet: pet não encontrado.
Linha 6 Coluna 24
Inserir
Windows:


Oracle SQL Developer : F:\MRDBChallengerPetCenter(Final)\PetCenterChallenger_Sprint4.sql
Arquivo Editar Exibir Navegar Executar Origem Equipe Ferramentas Janela Ajuda
Conexões
Oracle Conexões
FIAP_JULIA
Conexões do Serviço de Esquema de Banco de Dados
Relatórios
Todos os Relatórios
Administração de Banco de Dados
Application Express
ASH e AWR
Dicionário de Dados
PLSQL
Relatórios Definidos pelo Usuário
Relatórios de View Analítica
Relatórios do Data Modeler
Relatórios do Dicionário de Dados
Relatórios do TimesTen
Relatórios OLAP
Segurança
Sobre o Seu Banco de Dados
Streams
Tabela
Todos os Objetos
XML
Página de Boas-Vindas
PetCenterChallenger_Sprint4.sql
FIAP_JULIA
Planilha SQL
Histórico
4,48600006 segundos
Planilha
Query Builder
-- PET CENTER – SCRIPT COMPLETO (Sprints 1 a 4)
Saída do Script
Tarefa concluída em 4,486 segundos
Procedimento PL/SQL concluído com sucesso.
TESTE FUNÇÃO 2
Idade da Luna: 6 anos
TESTE DE EXCEÇÃO
ERRO fn_calcular_idade_pet: pet não encontrado.
Idade do pet inexistente: NULL
Procedimento PL/SQL concluído com sucesso.
PROCEDURE 1 - PETS EM JSON
JOIN -> Pet: Luna | Tutor: Ana Souza | Veterinário: Dra. Mariana Alves
JSON -> {"pet_id":1,"nome":"Luna","especie":"Cachorro","raca":"Shih-tzu","data_nascimento":"2020-05-10","tutor":"Ana Souza","veterinario":"Dra. Mariana Alves"}
JOIN -> Pet: Mimi | Tutor: Ana Souza | Veterinário: Dr. Felipe Costa
JSON -> {"pet_id":2,"nome":"Mimi","especie":"Gato","raca":"Siamês","data_nascimento":"2021-08-15","tutor":"Ana Souza","veterinario":"Dr. Felipe Costa"}
JOIN -> Pet: Thor | Tutor: Carlos Lima | Veterinário: Dra. Mariana Alves
JSON -> {"pet_id":3,"nome":"Thor","especie":"Cachorro","raca":"Golden Retriever","data_nascimento":"2019-03-20","tutor":"Carlos Lima","veterinario":"Dra. Mariana Alves"}
JOIN -> Pet: Rex | Tutor: Carlos Lima | Veterinário: Dr. Felipe Costa
JSON -> {"pet_id":4,"nome":"Rex","especie":"Cachorro","raca":"Pastor Alemão","data_nascimento":"2018-11-02","tutor":"Carlos Lima","veterinario":"Dr. Felipe Costa"}
JOIN -> Pet: Mel | Tutor: Beatriz Rocha | Veterinário: Dr. Ricardo Mendes
JSON -> {"pet_id":5,"nome":"Mel","especie":"Gato","raca":"Persa","data_nascimento":"2022-01-10","tutor":"Beatriz Rocha","veterinario":"Dr. Ricardo Mendes"}
Procedimento PL/SQL concluído com sucesso.
PROCEDURE 2 - REGISTROS AGRUPADOS
TIPO SUBTIPO VALOR
ALIMENTACAO AGUA 500.00
ALIMENTACAO RACAO 160.00
ALIMENTACAO NULL 660.00
ATIVIDADE EXERCICIO 2.00
ATIVIDADE SONO 10.00
ATIVIDADE NULL 12.00
COMPORTAMENTO COCEIRA 8.00
COMPORTAMENTO NULL 8.00
NULL NULL 680.00
Procedimento PL/SQL concluído com sucesso.
Linha 6 Coluna 24
Inserir
Windows:


Oracle SQL Developer : F:\MRDBChallengerPetCenter(Final)\PetCenterChallenger_Sprint4.sql
Arquivo Editar Exibir Navegar Executar Origem Equipe Ferramentas Janela Ajuda
Conexões
Oracle Conexões
FIAP_JULIA
Conexões do Serviço de Esquema de Banco de Dados
Relatórios
Todos os Relatórios
Administração de Banco de Dados
Application Express
ASH e AWR
Dicionário de Dados
PLSQL
Relatórios Definidos pelo Usuário
Relatórios de View Analítica
Relatórios do Data Modeler
Relatórios do Dicionário de Dados
Relatórios do TimesTen
Relatórios OLAP
Segurança
Sobre o Seu Banco de Dados
Streams
Tabela
Todos os Objetos
XML
Página de Boas-Vindas
PetCenterChallenger_Sprint4.sql
FIAP_JULIA
Planilha SQL Histórico
4,48600006 segundos
Planilha
Query Builder
-- PET CENTER – SCRIPT COMPLETO (Sprints 1 a 4)
Saída do Script
Tarefa concluída em 4,486 segundos
Package PKG_PETCENTER_AUDITORIA compilado
Package Body PKG_PETCENTER_AUDITORIA compilado
Trigger TRG_AUDITORIA_REGISTROS compilado
Trigger testada com sucesso: INSERT, UPDATE e DELETE.
Procedimento PL/SQL concluído com sucesso.
AUDITORIA DOS TESTES DA TRIGGER
ID: 1 | USUARIO: RM564975 | OPERACAO: INSERT | DATA: 2026-11-02 18:31:57 | OLD_ID: NULL | OLD_VALOR: NULL | OLD_ATUALIZADO_EM: NULL | NEW_ID: 9 | NEW_VALOR: 10 | NEW_ATUALIZADO_EM: NULL
ID: 2 | USUARIO: RM564975 | OPERACAO: UPDATE | DATA: 2026-11-02 18:31:57 | OLD_ID: 9 | OLD_VALOR: 10 | OLD_ATUALIZADO_EM: NULL | NEW_ID: 9 | NEW_VALOR: 20 | NEW_ATUALIZADO_EM: 2026-11-02 18:31:57
ID: 3 | USUARIO: RM564975 | OPERACAO: DELETE | DATA: 2026-11-02 18:31:57 | OLD_ID: 9 | OLD_VALOR: 20 | OLD_ATUALIZADO_EM: 2026-11-02 18:31:57 | NEW_ID: NULL | NEW_VALOR: NULL | NEW_ATUALIZADO_EM: NULL
Procedimento PL/SQL concluído com sucesso.
ID NOME ESPECIE RACA
1 Luna Cachorro Shih-tzu
2 Mimi Gato Siamês
3 Thor Cachorro Golden Retriever
4 Rex Cachorro Pastor Alemão
5 Mel Gato Persa
ID ENTRADA_ID TIPO SUBTIPO VALOR UNIDADE
8 2 ALIMENTACAO AGUA 500 ml
1 1 ALIMENTACAO RACAO 80 g
6 5 ALIMENTACAO RACAO 60 g
7 1 ALIMENTACAO RACAO 20 g
4 3 ATIVIDADE EXERCICIO 2 h
3 2 ATIVIDADE SONO 10 h
5 4 COMPORTAMENTO COCEIRA 8 episodios
7 linhas selecionadas.
ID AUDITORIA USUARIO BD TIPO OPERA DATA OPERACAO OLD TIPO OLD SUBTIPO
Linha 6 Coluna 24
Inserir
Windows:


Oracle SQL Developer : F:\MRDBChallengerPetCenter(Final)\PetCenterChallenger_Sprint4.sql
Arquivo Editar Exibir Navegar Executar Origem Equipe Ferramentas Janela Ajuda
Página de Boas-Vindas
PetCenterChallenger_Sprint4.sql
FIAP_JULIA
Planilha SQL Histórico
4,48600006 segundos
Planilha
Query Builder
-- PET CENTER – SCRIPT COMPLETO (Sprints 1 a 4)
Saída do Script
Tarefa concluída em 4,486 segundos
2 RM564975 UPDATE 02/11/26 18:31:57,913127000 TESTE_AUDITORIA TESTE 10
3 RM564975 DELETE 02/11/26 18:31:57,915438000 TESTE_AUDITORIA TESTE 20 02/1
===== EVIDENCIAS DE EXCECAO =====
1) pkg_petcenter_carga.pr_insert_users -> DUP_VAL_ON_INDEX (email duplicado)
LOG_ID=1 | PROCEDURE=pkg_petcenter_carga.pr_insert_users | USUARIO=RM564975 | DATA=2026-11-02 18:31:58 | SQLCODE=-1 | MSG=ORA-00001: restrição exclusiva (RM564975.SYS_C007387932) violada
2) pkg_petcenter_carga.pr_insert_users -> OTHERS (tipo_usuario inválido)
LOG_ID=2 | PROCEDURE=pkg_petcenter_carga.pr_insert_users | USUARIO=RM564975 | DATA=2026-11-02 18:31:58 | SQLCODE=-2290 | MSG=ORA-02290: restrição de verificação (RM564975.CHK_TIPO_USUARIO) violada
3) pkg_petcenter_carga.pr_insert_veterinarios -> OTHERS (user_id inexistente)
LOG_ID=3 | PROCEDURE=pkg_petcenter_carga.pr_insert_veterinarios | USUARIO=RM564975 | DATA=2026-11-02 18:31:58 | SQLCODE=-2291 | MSG=ORA-02291: restrição de integridade (RM564975.FK_VET_USER) violada - chave mãe não localizada
4) pkg_petcenter_carga.pr_insert_pets -> OTHERS (user_id inexistente)
LOG_ID=4 | PROCEDURE=pkg_petcenter_carga.pr_insert_pets | USUARIO=RM564975 | DATA=2026-11-02 18:31:58 | SQLCODE=-2291 | MSG=ORA-02291: restrição de integridade (RM564975.FK_PET_USER) violada - chave mãe não localizada
5) pkg_petcenter_carga.pr_insert_pet_veterinario -> OTHERS (pet_id inexistente)
LOG_ID=5 | PROCEDURE=pkg_petcenter_carga.pr_insert_pet_veterinario | USUARIO=RM564975 | DATA=2026-11-02 18:31:58 | SQLCODE=-2291 | MSG=ORA-02291: restrição de integridade (RM564975.FK_PV_PET) violada - chave mãe não localizada
6) pkg_petcenter_carga.pr_insert_diario_entradas -> DUP_VAL_ON_INDEX (pet+data duplicados)
LOG_ID=6 | PROCEDURE=pkg_petcenter_carga.pr_insert_diario_entradas | USUARIO=RM564975 | DATA=2026-11-02 18:31:58 | SQLCODE=-1 | MSG=ORA-00001: restrição exclusiva (RM564975.UQ_PET_DATA) violada
7) pkg_petcenter_carga.pr_insert_diario_entradas -> OTHERS (status inválido)
LOG_ID=7 | PROCEDURE=pkg_petcenter_carga.pr_insert_diario_entradas | USUARIO=RM564975 | DATA=2026-11-02 18:31:58 | SQLCODE=-2290 | MSG=ORA-02290: restrição de verificação (RM564975.CHK_DIARIO_STATUS) violada
8) pkg_petcenter_carga.pr_insert_registros -> OTHERS (entrada_id inexistente)
LOG_ID=8 | PROCEDURE=pkg_petcenter_carga.pr_insert_registros | USUARIO=RM564975 | DATA=2026-11-02 18:31:58 | SQLCODE=-2291 | MSG=ORA-02291: restrição de integridade (RM564975.FK_REGISTRO_ENTRADA) violada - chave mãe não localizada
9) pkg_petcenter_carga.pr_insert_comentarios -> OTHERS (entrada_id inexistente)
LOG_ID=9 | PROCEDURE=pkg_petcenter_carga.pr_insert_comentarios | USUARIO=RM564975 | DATA=2026-11-02 18:31:58 | SQLCODE=-2291 | MSG=ORA-02291: restrição de integridade (RM564975.FK_COMENTARIO_ENTRADA) violada - chave mãe não localizada
10) pkg_petcenter_carga.pr_insert_insights -> OTHERS (nivel_alerta inválido)
LOG_ID=10 | PROCEDURE=pkg_petcenter_carga.pr_insert_insights | USUARIO=RM564975 | DATA=2026-11-02 18:31:58 | SQLCODE=-2290 | MSG=ORA-02290: restrição de verificação (RM564975.CHK_NIVEL_ALERTA) violada
11) pkg_petcenter_carga.pr_insert_solicitacoes -> OTHERS (status inválido)
LOG_ID=11 | PROCEDURE=pkg_petcenter_carga.pr_insert_solicitacoes | USUARIO=RM564975 | DATA=2026-11-02 18:31:58 | SQLCODE=-2290 | MSG=ORA-02290: restrição de verificação (RM564975.CHK_SOL_STATUS) violada
12) pkg_petcenter_carga.pr_insert_alertas -> OTHERS (tipo inválido)
LOG_ID=12 | PROCEDURE=pkg_petcenter_carga.pr_insert_alertas | USUARIO=RM564975 | DATA=2026-11-02 18:31:58 | SQLCODE=-2290 | MSG=ORA-02290: restrição de verificação (RM564975.CHK_ALERTA_TIPO) violada
13) pkg_petcenter_relatorios.pr_relatorio_pets_json / pr_relatorio_registros_agrupado
NO_DATA_FOUND só ocorre com tabela vazia.
Não apagamos os dados da entrega para preservar a carga >= 5.
===== FIM DAS EVIDENCIAS DE EXCECAO =====
Linha 6 Coluna 24
Inserir
Windows:



```
Procedimento PL/SQL concluído com sucesso.

        ID NOME_PROCEDURE                                                                USUARIO_BD                                                                DATA_ERRO           CODIGO_ERRO
---------- ------------------------------------------------------------------------------ ------------------------------------------------------------------------- ------------------- -----------
MENSAGEM_ERRO
--------------------------------------------------------------------------------
         1 pkg_petcenter_carga.pr_insert_users                                            RM564975                                                                  2026-11-02 18:31:58          -1
ORA-00001: restrição exclusiva (RM564975.SYS_C007387932) violada

         2 pkg_petcenter_carga.pr_insert_users                                            RM564975                                                                  2026-11-02 18:31:58       -2290
ORA-02290: restrição de verificação (RM564975.CHK_TIPO_USUARIO) violada

         3 pkg_petcenter_carga.pr_insert_veterinarios                                     RM564975                                                                  2026-11-02 18:31:58       -2291
ORA-02291: restrição de integridade (RM564975.FK_VET_USER) violada - chave mãe não localizada


        ID NOME_PROCEDURE                                                                USUARIO_BD                                                                DATA_ERRO           CODIGO_ERRO
---------- ------------------------------------------------------------------------------ ------------------------------------------------------------------------- ------------------- -----------
MENSAGEM_ERRO
--------------------------------------------------------------------------------
         4 pkg_petcenter_carga.pr_insert_pets                                             RM564975                                                                  2026-11-02 18:31:58       -2291
ORA-02291: restrição de integridade (RM564975.FK_PET_USER) violada - chave mãe não localizada

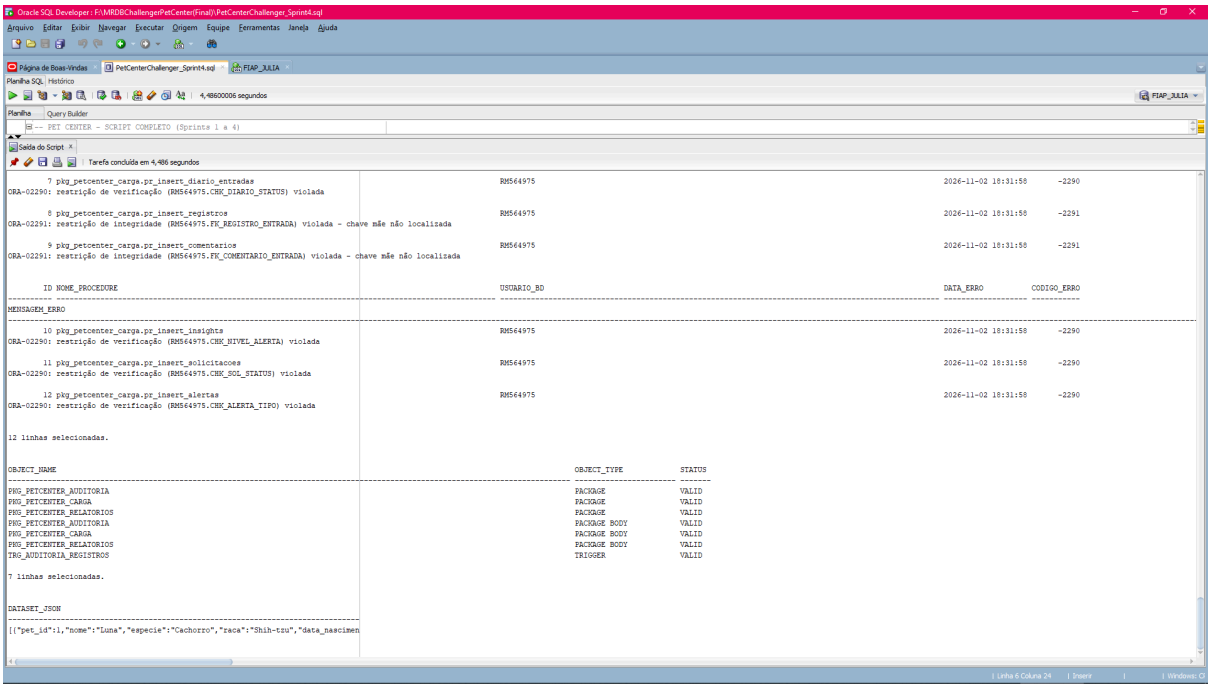
         5 pkg_petcenter_carga.pr_insert_pet_veterinario                                  RM564975                                                                  2026-11-02 18:31:58       -2291
ORA-02291: restrição de integridade (RM564975.FK_PV_PET) violada - chave mãe não localizada

         6 pkg_petcenter_carga.pr_insert_diario_entradas                                  RM564975                                                                  2026-11-02 18:31:58          -1
ORA-00001: restrição exclusiva (RM564975.UQ_PET_DATA) violada


        ID NOME_PROCEDURE                                                                USUARIO_BD                                                                DATA_ERRO           CODIGO_ERRO
---------- ------------------------------------------------------------------------------ ------------------------------------------------------------------------- ------------------- -----------
MENSAGEM_ERRO
--------------------------------------------------------------------------------
         7 pkg_petcenter_carga.pr_insert_diario_entradas                                  RM564975                                                                  2026-11-02 18:31:58       -2290
ORA-02290: restrição de verificação (RM564975.CHK_DIARIO_STATUS) violada

         8 pkg_petcenter_carga.pr_insert_registros                                        RM564975                                                                  2026-11-02 18:31:58       -2291
ORA-02291: restrição de integridade (RM564975.FK_REGISTRO_ENTRADA) violada - chave mãe não localizada

         9 pkg_petcenter_carga.pr_insert_comentarios                                      RM564975                                                                  2026-11-02 18:31:58       -2291
ORA-02291: restrição de integridade (RM564975.FK_COMENTARIO_ENTRADA) violada - chave mãe não localizada
```



```
         7 pkg_petcenter_carga.pr_insert_diario_entradas                                  RM564975                                                                  2026-11-02 18:31:58       -2290
ORA-02290: restrição de verificação (RM564975.CHK_DIARIO_STATUS) violada

         8 pkg_petcenter_carga.pr_insert_registros                                        RM564975                                                                  2026-11-02 18:31:58       -2291
ORA-02291: restrição de integridade (RM564975.FK_REGISTRO_ENTRADA) violada - chave mãe não localizada

         9 pkg_petcenter_carga.pr_insert_comentarios                                      RM564975                                                                  2026-11-02 18:31:58       -2291
ORA-02291: restrição de integridade (RM564975.FK_COMENTARIO_ENTRADA) violada - chave mãe não localizada


        ID NOME_PROCEDURE                                                                USUARIO_BD                                                                DATA_ERRO           CODIGO_ERRO
---------- ------------------------------------------------------------------------------ ------------------------------------------------------------------------- ------------------- -----------
MENSAGEM_ERRO
--------------------------------------------------------------------------------
        10 pkg_petcenter_carga.pr_insert_insights                                         RM564975                                                                  2026-11-02 18:31:58       -2290
ORA-02290: restrição de verificação (RM564975.CHK_NIVEL_ALERTA) violada

        11 pkg_petcenter_carga.pr_insert_solicitacoes                                     RM564975                                                                  2026-11-02 18:31:58       -2290
ORA-02290: restrição de verificação (RM564975.CHK_SOL_STATUS) violada

        12 pkg_petcenter_carga.pr_insert_alertas                                          RM564975                                                                  2026-11-02 18:31:58       -2290
ORA-02290: restrição de verificação (RM564975.CHK_ALERTA_TIPO) violada


12 linhas selecionadas.

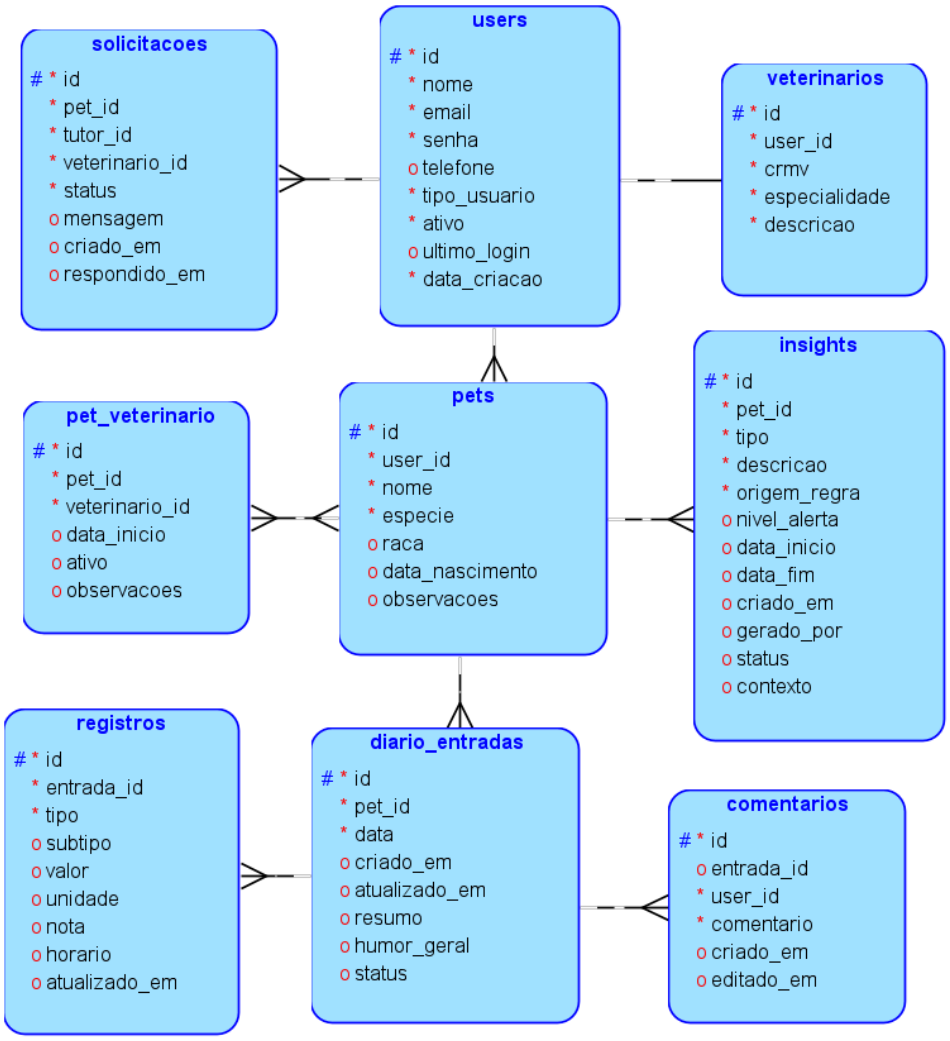

OBJECT_NAME                                                                                                                                                        OBJECT_TYPE         STATUS
-------------------------------------------------------------------------------------------------------------------------------------------------------------------- ------------------- -------
PKG_PETCENTER_AUDITORIA                                                                                                                                            PACKAGE             VALID
PKG_PETCENTER_CARGA                                                                                                                                                PACKAGE             VALID
PKG_PETCENTER_RELATORIOS                                                                                                                                           PACKAGE             VALID
PKG_PETCENTER_AUDITORIA                                                                                                                                            PACKAGE BODY        VALID
PKG_PETCENTER_CARGA                                                                                                                                                PACKAGE BODY        VALID
PKG_PETCENTER_RELATORIOS                                                                                                                                           PACKAGE BODY        VALID
TRG_AUDITORIA_REGISTROS                                                                                                                                            TRIGGER             VALID

7 linhas selecionadas.

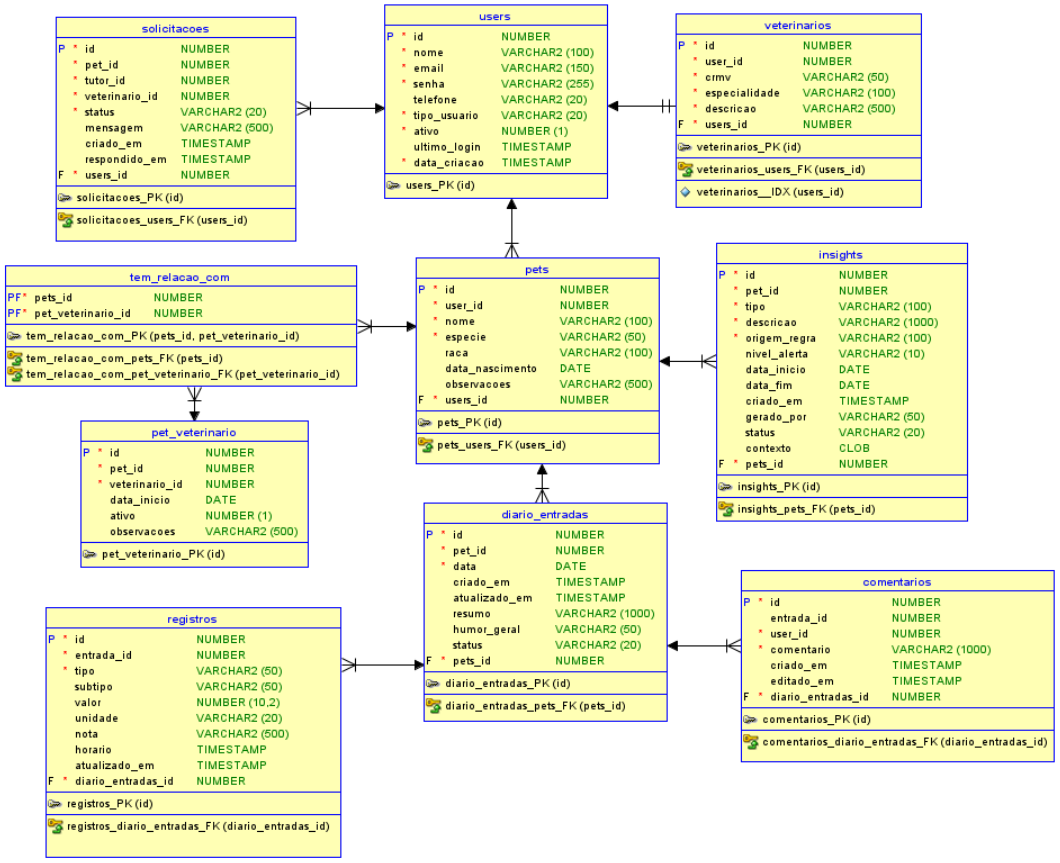

DATASET_JSON
--------------------------------------------------------------------------------
[{"pet_id":1,"nome":"Luna","especie":"Cachorro","raca":"Shih-tzu","data_nascimen
```

5 MODELAGENS (Oracle SQL Developer Data Modeler)

5.1 Modelo Lógico



5.2 Modelo Relacional



5.3 DDL original gerado a partir do modelo relacional (o sql foi posteriormente editado manualmente afim de atender os requisitos da ideia do projeto)

```
-- Gerado por Oracle SQL Developer Data Modeler 24.3.1.351.0831
--   em:        2026-05-22 21:05:40 BRT
--   site:      Oracle Database 11g
--   tipo:      Oracle Database 11g



-- predefined type, no DDL - MDSYS.SDO_GEOMETRY

-- predefined type, no DDL - XMLTYPE

CREATE TABLE comentarios
    (
     id                NUMBER  NOT NULL ,
     entrada_id          NUMBER ,
     user_id           NUMBER  NOT NULL ,
     comentario          VARCHAR2 (1000)  NOT NULL ,
```

```
    criado_em          TIMESTAMP ,
    editado_em         TIMESTAMP ,
    diario_entradas_id NUMBER  NOT NULL
    )
;

ALTER TABLE comentarios
   ADD CONSTRAINT comentarios_PK PRIMARY KEY ( id ) ;

CREATE TABLE diario_entradas
   (
    id            NUMBER  NOT NULL ,
    pet_id        NUMBER  NOT NULL ,
    data          DATE  NOT NULL ,
    criado_em     TIMESTAMP ,
    atualizado_em TIMESTAMP ,
    resumo        VARCHAR2 (1000) ,
    humor_geral   VARCHAR2 (50) ,
    status        VARCHAR2 (20) ,
    pets_id       NUMBER  NOT NULL
    )
;

ALTER TABLE diario_entradas
   ADD CONSTRAINT diario_entradas_PK PRIMARY KEY ( id ) ;

CREATE TABLE insights
   (
    id           NUMBER  NOT NULL ,
    pet_id       NUMBER  NOT NULL ,
    tipo         VARCHAR2 (100)  NOT NULL ,
    descricao    VARCHAR2 (1000)  NOT NULL ,
    origem_regra VARCHAR2 (100)  NOT NULL ,
    nivel_alerta VARCHAR2 (10) ,
    data_inicio  DATE ,
    data_fim     DATE ,
    criado_em    TIMESTAMP ,
    gerado_por   VARCHAR2 (50) ,
    status       VARCHAR2 (20) ,
    contexto     CLOB ,
    pets_id      NUMBER  NOT NULL
    )
;

ALTER TABLE insights
   ADD CONSTRAINT insights_PK PRIMARY KEY ( id ) ;

CREATE TABLE pet_veterinario
```

```
    (
     id            NUMBER  NOT NULL ,
     pet_id        NUMBER  NOT NULL ,
     veterinario_id NUMBER  NOT NULL ,
     data_inicio    DATE ,
     ativo         NUMBER (1) ,
     observacoes    VARCHAR2 (500)
    )
;

ALTER TABLE pet_veterinario
    ADD CONSTRAINT pet_veterinario_PK PRIMARY KEY ( id ) ;

CREATE TABLE pets
    (
     id             NUMBER  NOT NULL ,
     user_id        NUMBER  NOT NULL ,
     nome           VARCHAR2 (100)  NOT NULL ,
     especie        VARCHAR2 (50)  NOT NULL ,
     raca           VARCHAR2 (100) ,
     data_nascimento DATE ,
     observacoes     VARCHAR2 (500) ,
     users_id        NUMBER  NOT NULL
    )
;

ALTER TABLE pets
    ADD CONSTRAINT pets_PK PRIMARY KEY ( id ) ;

CREATE TABLE registros
    (
     id               NUMBER  NOT NULL ,
     entrada_id        NUMBER  NOT NULL ,
     tipo             VARCHAR2 (50)  NOT NULL ,
     subtipo           VARCHAR2 (50) ,
     valor            NUMBER (10,2) ,
     unidade           VARCHAR2 (20) ,
     nota             VARCHAR2 (500) ,
     horario           TIMESTAMP ,
     atualizado_em      TIMESTAMP ,
     diario_entradas_id NUMBER  NOT NULL
    )
;

ALTER TABLE registros
    ADD CONSTRAINT registros_PK PRIMARY KEY ( id ) ;

CREATE TABLE solicitacoes
```

```
    (
     id            NUMBER  NOT NULL ,
     pet_id        NUMBER  NOT NULL ,
     tutor_id      NUMBER  NOT NULL ,
     veterinario_id NUMBER  NOT NULL ,
     status        VARCHAR2 (20)  NOT NULL ,
     mensagem      VARCHAR2 (500) ,
     criado_em     TIMESTAMP ,
     respondido_em  TIMESTAMP ,
     users_id      NUMBER  NOT NULL
    )
;

ALTER TABLE solicitacoes
    ADD CONSTRAINT solicitacoes_PK PRIMARY KEY ( id ) ;

CREATE TABLE tem_relacao_com
    (
     pets_id           NUMBER  NOT NULL ,
     pet_veterinario_id NUMBER  NOT NULL
    )
;

ALTER TABLE tem_relacao_com
    ADD CONSTRAINT tem_relacao_com_PK PRIMARY KEY ( pets_id, pet_veterinario_id ) ;

CREATE TABLE users
    (
     id          NUMBER  NOT NULL ,
     nome        VARCHAR2 (100)  NOT NULL ,
     email       VARCHAR2 (150)  NOT NULL ,
     senha       VARCHAR2 (255)  NOT NULL ,
     telefone    VARCHAR2 (20) ,
     tipo_usuario VARCHAR2 (20)  NOT NULL ,
     ativo       NUMBER (1)  NOT NULL ,
     ultimo_login TIMESTAMP ,
     data_criacao TIMESTAMP  NOT NULL
    )
;

ALTER TABLE users
    ADD CONSTRAINT users_PK PRIMARY KEY ( id ) ;

CREATE TABLE veterinarios
    (
     id          NUMBER  NOT NULL ,
     user_id     NUMBER  NOT NULL ,
     crmv        VARCHAR2 (50)  NOT NULL ,
```

```
    especialidade VARCHAR2 (100)  NOT NULL ,
    descricao     VARCHAR2 (500)  NOT NULL ,
    users_id      NUMBER  NOT NULL
   )
;
CREATE UNIQUE INDEX veterinarios__IDX ON veterinarios
   (
    users_id ASC
   )
;

ALTER TABLE veterinarios
   ADD CONSTRAINT veterinarios_PK PRIMARY KEY ( id ) ;

ALTER TABLE comentarios
   ADD CONSTRAINT comentarios_diario_entradas_FK FOREIGN KEY
   (
    diario_entradas_id
   )
   REFERENCES diario_entradas
   (
    id
   )
;

ALTER TABLE diario_entradas
   ADD CONSTRAINT diario_entradas_pets_FK FOREIGN KEY
   (
    pets_id
   )
   REFERENCES pets
   (
    id
   )
;

ALTER TABLE insights
   ADD CONSTRAINT insights_pets_FK FOREIGN KEY
   (
    pets_id
   )
   REFERENCES pets
   (
    id
   )
;

ALTER TABLE pets
```

```
    ADD CONSTRAINT pets_users_FK FOREIGN KEY
    (
     users_id
    )
    REFERENCES users
    (
     id
    )
;

ALTER TABLE registros
    ADD CONSTRAINT registros_diario_entradas_FK FOREIGN KEY
    (
     diario_entradas_id
    )
    REFERENCES diario_entradas
    (
     id
    )
;

ALTER TABLE solicitacoes
    ADD CONSTRAINT solicitacoes_users_FK FOREIGN KEY
    (
     users_id
    )
    REFERENCES users
    (
     id
    )
;

--  ERROR: FK name length exceeds maximum allowed length(30)
ALTER TABLE tem_relacao_com
    ADD CONSTRAINT tem_relacao_com_pet_veterinario_FK FOREIGN KEY
    (
     pet_veterinario_id
    )
    REFERENCES pet_veterinario
    (
     id
    )
;

ALTER TABLE tem_relacao_com
    ADD CONSTRAINT tem_relacao_com_pets_FK FOREIGN KEY
    (
     pets_id
```

```
    )
    REFERENCES pets
    (
     id
    )
;

ALTER TABLE veterinarios
    ADD CONSTRAINT veterinarios_users_FK FOREIGN KEY
    (
     users_id
    )
    REFERENCES users
    (
     id
    )
;



-- Relatório do Resumo do Oracle SQL Developer Data Modeler:
--
-- CREATE TABLE                            10
-- CREATE INDEX                             1
-- ALTER TABLE                             19
-- CREATE VIEW                              0
-- ALTER VIEW                              0
-- CREATE PACKAGE                           0
-- CREATE PACKAGE BODY                      0
-- CREATE PROCEDURE                         0
-- CREATE FUNCTION                          0
-- CREATE TRIGGER                           0
-- ALTER TRIGGER                           0
-- CREATE COLLECTION TYPE                   0
-- CREATE STRUCTURED TYPE                   0
-- CREATE STRUCTURED TYPE BODY              0
-- CREATE CLUSTER                           0
-- CREATE CONTEXT                           0
-- CREATE DATABASE                          0
-- CREATE DIMENSION                         0
-- CREATE DIRECTORY                         0
-- CREATE DISK GROUP                        0
-- CREATE ROLE                             0
-- CREATE ROLLBACK SEGMENT                  0
-- CREATE SEQUENCE                          0
-- CREATE MATERIALIZED VIEW                 0
-- CREATE MATERIALIZED VIEW LOG             0
-- CREATE SYNONYM                           0
```

-- CREATE TABLESPACE 0
-- CREATE USER 0
--
-- DROP TABLESPACE 0
-- DROP DATABASE 0
--
-- REDACTION POLICY 0
--
-- ORDS DROP SCHEMA 0
-- ORDS ENABLE SCHEMA 0
-- ORDS ENABLE OBJECT 0
--
-- ERRORS 1
-- WARNINGS 0